

1 Computational Geometry

1.1 basic

```

1 struct point {
2     double x, y;
3     point(double x = 0, double y = 0) : x(x), y(y) {}
4     point operator - (const point& b) const { return point(x - b.x, y - b.y); }
5     point operator + (const point& b) const { return point(x + b.x, y + b.y); }
6     point operator * (double k) const { return point(x * k, y * k); }
7     bool operator < (const point& b) { if (x != b.x) return x < b.x; return y < b.y; }
8 };
9
10 double cross(point a, point b) {return a.x * b.y - a.y * b.x;}
11 // 直線ab與直線cd的交點
12 point intersect(point a, point b, point c, point d)
13 {
14     double c1 = cross(b - a, c - a);
15     double c2 = cross(b - a, d - a);
16     return c + (d - c) * (c1 / (c1 - c2));
17 }

```

1.2 convex-hull

```

1 double cross(point a, point b) {return a.x * b.y - a.y * b.x;}
2 vector<point> build(vector<point> v)
3 {
4     vector<point> ans;
5     sort(v.begin(), v.end());
6     for(int i = 0; i < v.size(); i++){
7         while(ans.size() > 1 and cross(ans.end()[-2] - ans.end()[-1], ans.end()[-2] - v[i]) > 0) ans.pop_back();
8         ans.pb(v[i]);
9     }
10    for(int i = v.size() - 2; i >= 0; i--){
11        while(ans.size() > 1 and cross(ans.end()[-2] - ans.end()[-1], ans.end()[-2] - v[i]) > 0) ans.pop_back();
12        ans.pb(v[i]);
13    }
14    ans.pop_back();
15    return ans;
16 }

```

1.3 Line Segment Intersection

```

1 #include<bits/stdc++.h>
2 using namespace std;
3 #define KCC ios::sync_with_stdio(0);cin.tie(0);
4 #define pb push_back
5 #define pii pair<int, int>
6 #define F first
7 #define S second
8 #define int long long
9
10 pii f(pii a, pii b)
11 {
12     return {b.F - a.F, b.S - a.S};
13 }
14 int across(pii a, pii b)
15 {
16     if(a.F * b.S < a.S * b.F) return -1;
17     if(a.F * b.S > a.S * b.F) return 1;
18     return 0;
19 }
20 bool overlap(pii a, pii b, pii c, pii d)
21 {
22     if(a.F == b.F and b.F == c.F and c.F == d.F){
23         if(min(a.S, b.S) > max(c.S, d.S) or min(c.S, d.S) > max(a.S, b.S)) return false;
24     }
25     if(min(a.F, b.F) > max(c.F, d.F) or min(c.F, d.F) > max(a.F, b.F)) return false;
26     return true;
27 }
28 void solve()
29 {
30     int x1, x2, x3, x4, y1, y2, y3, y4;
31     cin >> x1 >> y1 >> x2 >> y2 >> x3 >> y3 >> x4 >> y4;
32     pii a = {x1, y1};
33     pii b = {x2, y2};
34     pii c = {x3, y3};
35     pii d = {x4, y4};
36     int x = across(f(a, b), f(a, c));
37     int y = across(f(a, b), f(a, d));
38     int z = across(f(c, d), f(c, a));
39     int w = across(f(c, d), f(c, b));
40     if(x * y < 0 and z * w < 0) cout << "YES\n";
41     else if(x * y <= 0 and z * w <= 0 and overlap(a, b, c, d)) cout << "YES\n";
42     else cout << "NO\n";
43 }
44 signed main()
45 {
46     KCC
47     int t;
48     cin >> t;
49     while(t--) solve();
50     return 0;
51 }

```

1.4 Point in Polygon

```

1 #include<bits/stdc++.h>
2 using namespace std;
3 #define KCC ios::sync_with_stdio(0);cin.tie(0);
4 #define pb push_back
5 #define pii pair<long long, long long>
6 #define F first
7 #define S second
8 #define int long long
9
10 int x[(int)1e3 + 5], y[(int) 1e3 + 5], n, m;
11
12 bool onsegment(pii a, pii b, pii p)
13 {
14     if((a.F - b.F) * (a.S - p.S) != (a.F - p.F) * (a.S - b.S)) return false;
15     if(!(min(a.F, b.F) <= p.F and p.F <= max(a.F, b.F))) return false;
16     if(!(min(a.S, b.S) <= p.S and p.S <= max(a.S, b.S))) return false;
17     return true;
18 }
19
20 signed main()
21 {
22     KCC
23     cin >> n >> m;
24     for(int i = 0; i < n; i++) cin >> x[i] >> y[i];
25     while(m--){
26         int a, b;
27         cin >> a >> b;
28         pii p = {a, b};
29         bool boundary = false;
30         int cnt = 0;
31         for(int i = 0; i < n; i++){
32             pii u = {x[i], y[i]}, v = {x[(i + 1) % n], y[(i + 1) % n]};
33             if(onsegment(u, v, p)){
34                 boundary = true;
35                 break;
36             }
37             if(min(u.S, v.S) <= b and b < max(u.S, v.S)){
38                 /*
39                 Line u_to_v:
40                 X = u.F + t(v.F - u.F)
41                 Y = u.S + t(v.S - u.S)
42                 b = u.S + t(v.S - u.S)
43                 t = (b - u.S) / (v.S - u.S)
44                 X = u.F + (b - u.S) / (v.S - u.S) * (v.F - u.F)
45                 */
46                 double X = u.F + 1.0 * (b - u.S) / (v.S - u.S) * (v.F - u.F);
47                 if(a <= X) cnt++;
48             }
49         }
50         if (boundary) cout << "BOUNDARY\n";
51         else if (cnt & 1) cout << "INSIDE\n";
52         else cout << "OUTSIDE\n";
53     }
54 }

```

1.5 動態開點李超樹

```

1 struct Line{
2     int k, b;
3     int val(int x){
4         return k * x + b;
5     }
6 };
7 struct Node{
8     Node *l = nullptr;
9     Node *r = nullptr;
10    bool has_line = false;
11    Line line;
12 } *root = nullptr; //維護最大值
13 void insert(Node *&node, int l, int r, Line new_line)
14 {
15     if(!node) node = new Node();
16     if(!node->has_line){
17         node->line = new_line;
18         node->has_line = true;
19         return;
20     }
21     int mid = (l + r) >> 1;
22     bool l_better = new_line.val(l) > node->line.val(l);
23     bool mid_better = new_line.val(mid) > node->line.val(mid);
24     if(mid_better) swap(new_line, node->line);
25     if(l == r) return;
26     if(l_better != mid_better) insert(node->l, l, mid, new_line);
27     else insert(node->r, mid + 1, r, new_line);
28 }
29 int query(Node *node, int l, int r, int x)
30 {
31     if(!node) return -2e18;
32     if(!node->has_line) return -2e18;
33     int re = node->line.val(x);
34     if(l == r) return re;
35     int mid = (l + r) >> 1;
36     if(x <= mid) re = max(re, query(node->l, l, mid, x));
37     else re = max(re, query(node->r, mid + 1, r, x));
38     return re;
39 }

```

1.6 半平面交集

```

1 // 用有向線段 AB 切割多邊形 v，保留左半邊
2 vector<point> cut(vector<point> v, point a, point b)
3 {
4     vector<point> ans;
5     int n = v.size();
6
7     for(int i = 0; i < n; i++){
8         point c = v[i], d = v[(i + 1) % n];
9     }

```

```

10 double side_c = cross(b - a, c - a);
11 double side_d = cross(b - a, d - a);
12 if(side_c >= 0){
13     ans.pb(c);
14     if(side_d < 0) ans.pb(intersect(
15         a, b, c, d));
16 }
17 else{
18     if(side_d >= 0) ans.pb(intersect(
19         a, b, c, d));
20 }
21 }
22 return ans;
23 }

```

1.7 最近點對

```

1 #include<bits/stdc++.h>
2 using namespace std;
3 #define KCC ios::sync_with_stdio(0);cin.tie
4 (0);
5 #define pb push_back
6 #define pii pair<long long, long long>
7 #define F first
8 #define S second
9 #define int long long
10 pii arr[(int)2e5 + 5];
11 int dis(int i, pii w)
12 {
13     return (arr[i].F - w.S) * (arr[i].F - w.
14         S) + (arr[i].S - w.F) * (arr[i].S -
15         w.F);
16 }
17 signed main()
18 {
19     KCC
20     int n, ans = 8e18, id = 0;
21     cin >> n;
22     for(int i = 0; i < n; i++) cin >> arr[i]
23         .F >> arr[i].S;
24     sort(arr, arr + n);
25     set<pii> s; // {y, x}
26     for(int i = 0; i < n; i++){
27         int d = sqrt(ans) + 1;
28         while(s.size() > 0 and arr[i].F -
29             arr[id].F >= d){
30             s.erase({arr[id].S, arr[id].F});
31             id++;
32         }
33         auto l = s.lower_bound({arr[i].S - d
34             , -1e9});
35         auto r = s.lower_bound({arr[i].S + d
36             , 1e9});
37         for(auto j = l; j != r; ++j) ans =
38             min(ans, dis(i, *j));
39         s.insert({arr[i].S, arr[i].F});
40     }
41     cout << ans << "\n";
42     return 0;
43 }

```

1.8 李超線段樹

```

1 struct Line{
2     int k, b;
3     int val(int x){
4         return k * x + b;
5     }
6 };
7 Line tree[(int)1e6]; //維護最大值
8 bool has_line[(int)1e6];
9 void insert(int node, int l, int r, Line
10     new_line)
11 {
12     if(!has_line[node]){
13         tree[node] = new_line;
14         has_line[node] = true;
15         return;
16     }
17     int mid = (l + r) >> 1;
18     bool l_better = new_line.val(l) > tree[
19         node].val(l);
20     bool mid_better = new_line.val(mid) >
21         tree[node].val(mid);
22     if(mid_better) swap(new_line, tree[node
23         ]);
24     if(l == r) return;
25     if(l_better != mid_better) insert(node
26         << 1, l, mid, new_line);
27     else insert(node << 1 | 1, mid + 1, r,
28         new_line);
29 }
30 int query(int node, int l, int r, int x)
31 {
32     if(!has_line[node]) return -2e18;
33     int re = tree[node].val(x);
34     if(l == r) return re;
35     int mid = (l + r) >> 1;
36     if(x <= mid) re = max(re, query(node <<
37         1, l, mid, x));
38     else re = max(re, query(node << 1 | 1,
39         mid + 1, r, x));
40     return re;
41 }

```

1.9 矩形覆蓋面積

```

1 #include<bits/stdc++.h>
2 using namespace std;
3 #define KCC ios::sync_with_stdio(0);cin.tie
4 (0);
5 #define pb push_back
6 #define pii pair<long long, long long>
7 #define F first
8 #define S second
9 #define int long long
10 struct SEG{
11     int cnt, len;
12 }seg[(int)1e7];
13 void pull(int id, int l, int r)
14 {

```

```

15 if(seg[id].cnt > 0) seg[id].len = r - l
16     + 1;
17 else{
18     if(l != r) seg[id].len = seg[id <<
19         1].len + seg[id << 1 | 1].len;
20     else seg[id].len = 0;
21 }
22 void update(int ql, int qr, int x, int l,
23     int r, int id)
24 {
25     if(ql <= l and r <= qr) {
26         seg[id].cnt += x;
27         pull(id, l, r);
28         return;
29     }
30     int mid = (l + r) >> 1;
31     if(ql <= mid) update(ql, qr, x, l, mid,
32         id << 1);
33     if(qr > mid) update(ql, qr, x, mid + 1,
34         r, id << 1 | 1);
35     pull(id, l, r);
36 }
37 signed main()
38 {
39     KCC
40     int n, ans = 0;
41     cin >> n;
42     vector<array<int, 4>> v;
43     for(int i = 0; i < n; i++){
44         int a, b, c, d;
45         cin >> a >> b >> c >> d;
46         a += 1e6 + 1;
47         b += 1e6 + 1;
48         c += 1e6 + 1;
49         d += 1e6 + 1;
50         v.pb({a, b, d, 1});
51         v.pb({c, b, d, -1});
52     }
53     sort(v.begin(), v.end());
54     int last = v[0][0], maxval = 2e6 + 1;
55     for(auto [x, y1, y2, sign] : v){
56         ans += (x - last) * seg[1].len;
57         update(y1, y2 - 1, sign, 1, maxval,
58             1);
59         last = x;
60     }
61     cout << ans << "\n";
62     return 0;
63 }

```

2 Data Structure

2.1 DSU-rollback

```

1 struct DSU_rollback{
2     int components;
3     vector<int> parent, sz;
4     struct modify{
5         int u, v, pre_sz;
6     };

```

```

7     vector<modify> history;
8     void initial(int n){
9         parent.resize(n + 1);
10        sz.resize(n + 1, 1);
11        components = n;
12        for(int i = 1; i <= n; i++) parent[i]
13            = i;
14    }
15    int find(int x){
16        while(x != parent[x]) x = parent[x];
17        return x;
18    }
19    void unite(int u, int v){
20        int root_u = find(u), root_v = find(
21            v);
22        if(root_u == root_v) return;
23        if(sz[root_u] < sz[root_v]) swap(
24            root_u, root_v);
25        history.pb({root_u, root_v, sz[
26            root_u]});
27        parent[root_v] = root_u;
28        sz[root_u] += sz[root_v];
29        components--;
30    }
31    void rollback(){
32        if(history.size() == 0) return;
33        modify last = history.back();
34        history.pop_back();
35        parent[last.v] = last.v;
36        sz[last.u] = last.pre_sz;
37        components++;
38    }
39    int dsutime(){return (int)history.size()
40        ;}
41    void return_to_t(int t){
42        while(history.size() > t) rollback()
43            ;
44    }
45 };

```

2.2 FHQ-treap

```

1 #include<bits/stdc++.h>
2 using namespace std;
3 #define KCC ios::sync_with_stdio(0);cin.tie
4 (0);
5 #define pb push_back
6 #define pii pair<int, int>
7 #define F first
8 #define S second
9 #define int long long
10 const int MAXN = 2e5 + 5;
11 int left_node[MAXN], right_node[MAXN],
12     priority[MAXN], sz[MAXN], node_cnt = 0,
13     root = 0;
14 char val[MAXN];
15 void up(int i)
16 {
17     sz[i] = sz[left_node[i]] + sz[right_node
18         [i]] + 1;
19 }
20 void split(int i, int l, int r, int rank)
21 {

```

2.3 persistent-segment-tree

```

18 if(i == 0){
19     left_node[r] = right_node[l] = 0;
20     return;
21 }
22 if(sz[left_node[i]] + 1 <= rank){
23     right_node[l] = i;
24     split(right_node[i], i, r, rank - sz
25           [left_node[i]] - 1);
26 }
27 else{
28     left_node[r] = i;
29     split(left_node[i], l, i, rank);
30 }
31 up(i);
32 int merge(int l, int r)
33 {
34     if(l == 0 or r == 0) return l + r;
35     if(priority[l] >= priority[r]){
36         right_node[l] = merge(right_node[l],
37                               r);
38         up(l);
39         return l;
40     }
41     else{
42         left_node[r] = merge(l, left_node[r]
43                             );
44         up(r);
45         return r;
46     }
47 }
48 void out(int i)
49 {
50     if(i == 0) return;
51     out(left_node[i]);
52     cout << val[i];
53     out(right_node[i]);
54 }
55 int main()
56 {
57     KCC
58     int n, m;
59     cin >> n >> m;
60     for(int i = 0; i < n; i++){
61         char c;
62         cin >> c;
63         priority[++node_cnt] = rnd();
64         val[node_cnt] = c;
65         sz[node_cnt] = 1;
66         root = merge(root, node_cnt);
67     }
68     while(m--){
69         int a, b;
70         cin >> a >> b;
71         split(root, 0, 0, b);
72         int lm = right_node[0];
73         int r = left_node[0];
74         split(lm, 0, 0, a - 1);
75         int l = right_node[0];
76         int m = left_node[0];
77         root = merge(merge(l, r), m);
78     }
79     out(root);
80     return 0;

```

```

1 int upd(int pre, int l, int r, int pos, int
2   val)
3 {
4     int node = ++cnt_tree;
5     tree[node] = tree[pre];
6     if(l == r){
7         tree[node].sum += val;
8         return node;
9     }
10    int mid = (l + r) >> 1;
11    if(pos <= mid) tree[node].l = upd(tree[
12      pre].l, l, mid, pos, val);
13    else tree[node].r = upd(tree[pre].r, mid
14      + 1, r, pos, val);
15    tree[node].sum = tree[tree[node].l].sum
16      + tree[tree[node].r].sum;
17    return node;
18 }
19 int query(int node, int l, int r, int ql,
20   int qr)
21 {
22     if(ql <= l and r <= qr) return tree[node
23       ].sum;
24     int mid = (l + r) >> 1, re = 0;
25     if(ql <= mid) re += query(tree[node].l,
26       l, mid, ql, qr);
27     if(qr > mid) re += query(tree[node].r,
28       mid + 1, r, ql, qr);
29     return re;
30 }
31 int get_kth(int node_l, int node_r, int l,
32   int r, int k)
33 {
34     //原陣列位置區間 (node_l, node_r] 第 k 小
35     /*
36     if(l == r) return l;
37     int mid = (l + r) >> 1;
38     int left_cnt = tree[tree[node_r].l].sum
39       - tree[tree[node_l].l].sum;
40     if(k <= left_cnt) return get_kth(tree[
41       node_l].l, tree[node_r].l, l, mid, k
42       );
43     else return get_kth(tree[node_l].r, tree
44       [node_r].r, mid + 1, r, k - left_cnt
45       );
46     */
47     while (l < r) {
48         int mid = (l + r) >> 1;
49         int left_cnt = tree[tree[node_r].l].
50           sum - tree[tree[node_l].l].sum;
51         if (k <= left_cnt) {
52             node_l = tree[node_l].l;
53             node_r = tree[node_r].l;
54             r = mid;
55         }
56         else {
57             k -= left_cnt;
58             node_l = tree[node_l].r;
59             node_r = tree[node_r].r;
60             l = mid + 1;
61         }
62     }
63     return l;
64 }

```

2.4 segment tree lazytag

```

1 struct SegmentTree{
2     struct Node{
3         int sum, f, d, len;
4     } tree[MAXN << 2];
5     void pull(int id){
6         tree[id].sum = tree[id << 1].sum +
7           tree[id << 1 | 1].sum;
8     }
9     void addtag(int id, int a, int d){
10        tree[id].f += a;
11        tree[id].d += d;
12        tree[id].sum += (2 * a + (tree[id].
13          len - 1) * d) * tree[id].len /
14          2;
15    }
16    void push(int id){
17        if(!tree[id].f && !tree[id].d)
18            return;
19        int mid_f = tree[id].f + tree[id <<
20          1].len * tree[id].d;
21        addtag(id << 1, tree[id].f, tree[id
22          ].d);
23        addtag(id << 1 | 1, mid_f, tree[id].
24          d);
25        tree[id].f = tree[id].d = 0;
26    }
27    void build(int l, int r, int id, int ar
28      []){
29        tree[id].len = r - l + 1;
30        tree[id].f = tree[id].d = 0;
31        if(l == r){
32            tree[id].sum = ar[l];
33            return;
34        }
35        int mid = (l + r) >> 1;
36        build(l, mid, id << 1, ar);
37        build(mid + 1, r, id << 1 | 1, ar);
38        pull(id);
39    }
40    void update(int ql, int qr, int a, int d
41      , int l, int r, int id){
42        if(ql <= l and r <= qr){
43            addtag(id, a + (l - ql) * d, d);
44            return;
45        }
46        push(id);
47        int mid = (l + r) >> 1;
48        if(ql <= mid) update(ql, qr, a, d, l
49          , mid, id << 1);
50        if(qr > mid) update(ql, qr, a, d,
51          mid + 1, r, id << 1 | 1);
52        pull(id);
53    }
54    int query(int ql, int qr, int l, int r,
55      int id){
56        if(ql <= l and r <= qr) return tree[
57          id].sum;
58        push(id);
59        int mid = (l + r) >> 1, re = 0;
60        if(ql <= mid) re += query(ql, qr, l,
61          mid, id << 1);
62        if(qr > mid) re += query(ql, qr, mid
63          + 1, r, id << 1 | 1);

```

```

49     return re;
50 }
51 };

```

2.5 trie

```

1 struct Node{
2     int next_node[2]; // 01-trie
3     int max_idx;
4     Node() {
5         next_node[0] = next_node[1] = -1;
6         max_idx = -1;
7     }
8 };
9 vector<Node> trie;
10 void init_trie()
11 {
12     trie.clear();
13     trie.emplace_back(); // 插入根節點
14 }
15 void insert(int x, int id)
16 {
17     int u = 0;
18     for(int i = 29; i >= 0; i--){
19         int bit = (x >> i) & 1;
20         if(trie[u].next_node[bit] == -1){
21             trie[u].next_node[bit] = trie.
22               size();
23             trie.emplace_back();
24         }
25         u = trie[u].next_node[bit];
26         trie[u].max_idx = max(trie[u].
27           max_idx, id);
28     }
29 }
30 // 減少空間的存法
31 struct Edge {
32     int to, nxt;
33     char c;
34 };
35 struct Node {
36     int head = -1;
37     // 放節點要存的資訊
38 };
39 vector<Node> trie;
40 vector<Edge> edges;
41 int get_child(int u, char c){
42     for(int e = trie[u].head; e != -1; e =
43       edges[e].nxt){
44         if(edges[e].c == c) return edges[e].
45           to;
46     }
47     return -1;
48 }
49 int nxt_child(int u, char c){
50     int v = get_child(u, c);
51     if(v != -1) return v;
52     v = trie.size();
53     trie.emplace_back();

```

```

52 |
53 | edges.push_back({v, trie[u].head, c});
54 | trie[u].head = (int)edges.size() - 1;
55 | return v;
56 | }

```

2.6 Weighted DSU

```

1 | struct Weighted_DSU{
2 |     vector<int> parent, dis;
3 |     void initial(int n){
4 |         parent.resize(n + 1);
5 |         dis.resize(n + 1, 0);
6 |         for(int i = 1; i <= n; i++) parent[i] = i;
7 |     }
8 |     int find(int x){
9 |         if(parent[x] != x){
10 |             int tmp = parent[x];
11 |             parent[x] = find(tmp);
12 |             dis[x] += dis[tmp];
13 |         }
14 |         return parent[x];
15 |     }
16 |     void unite(int l, int r, int d){
17 |         int lf = find(l), rf = find(r);
18 |         if(lf != rf){
19 |             parent[lf] = rf;
20 |             dis[lf] = dis[r] - dis[l] + d;
21 |         }
22 |     }
23 |     int query(int l, int r){
24 |         if(find(l) != find(r)) return 1e18;
25 |         return dis[l] - dis[r];
26 |     }
27 | };

```

2.7 動態開點線段樹

```

1 | struct Node{
2 |     int sum = 0;
3 |     Node *l = nullptr;
4 |     Node *r = nullptr;
5 | }*root = nullptr;
6 |
7 | void add(Node *&node, int L, int R, int pos, int val)
8 | {
9 |     if(!node) node = new Node();
10 |    node->sum += val;
11 |    if(L == R) return;
12 |    int mid = (L + R) / 2;
13 |    if(pos <= mid) add(node->l, L, mid, pos, val);
14 |    else add(node->r, mid + 1, R, pos, val);
15 | }
16 | int query(Node *node, int L, int R, int ql, int qr)
17 | {
18 |     if(!node) return 0;

```

```

19 |     if(ql <= L and R <= qr) return node->sum;
20 |     ;
21 |     int mid = (L + R) / 2, ans = 0;
22 |     if(ql <= mid) ans += query(node->l, L, mid, ql, qr);
23 |     if(qr > mid) ans += query(node->r, mid + 1, R, ql, qr);
24 |     return ans;

```

3 Flow

3.1 dinic

```

1 | struct Dinic{
2 |     struct Edge{
3 |         int to, cap, rev;
4 |     };
5 |     vector<vector<Edge>> adj;
6 |     vector<int> level, ptr;
7 |     int n;
8 |     Dinic(int _n){
9 |         n = _n;
10 |        level.resize(n + 1);
11 |        ptr.resize(n + 1);
12 |        adj.resize(n + 1);
13 |    }
14 |    void addedge(int u, int v, int w){
15 |        adj[u].pb({v, w, adj[v].size()});
16 |        adj[v].pb({u, 0, adj[u].size() - 1});
17 |    }
18 |    bool bfs(int s, int t)
19 |    {
20 |        fill(level.begin(), level.end(), -1);
21 |        ;
22 |        level[s] = 0;
23 |        queue<int> q;
24 |        q.push(s);
25 |        while(q.size()){
26 |            int x = q.front(); q.pop();
27 |            for(auto edge : adj[x]){
28 |                if(edge.cap > 0 and level[edge.to] == -1){
29 |                    level[edge.to] = level[x] + 1;
30 |                    q.push(edge.to);
31 |                }
32 |            }
33 |        }
34 |        return (level[t] != -1);
35 |    }
36 |    int dfs(int now, int t, int pushed)
37 |    {
38 |        if(now == t) return pushed;
39 |        for(int &i = ptr[now]; i < adj[now].size(); i++){
40 |            auto &w = adj[now][i];
41 |            if(level[now] + 1 != level[w.to] or w.cap == 0) continue;

```

```

42 |            int tr = dfs(w.to, t, min(pushed, w.cap));
43 |            if(tr == 0) continue;
44 |            w.cap -= tr;
45 |            adj[w.to][w.rev].cap += tr;
46 |            return tr;
47 |        }
48 |        return 0;
49 |    }
50 |    int compute_max_flow(){
51 |        int max_flow = 0;
52 |        while(bfs(1, n)){
53 |            fill(ptr.begin(), ptr.end(), 0);
54 |            while(int pushed = dfs(1, n, 1e18)){
55 |                max_flow += pushed;
56 |            }
57 |        }
58 |        return max_flow;
59 |    }

```

3.2 dinic flow decomposition

```

1 | struct dinic{
2 |     struct edge{
3 |         int to, cap, rev;
4 |     };
5 |     vector<vector<edge>> adj;
6 |     vector<int> level, ptr;
7 |     int n;
8 |     vector<vector<int>> fwd;
9 |     dinic(int _n){
10 |         n = _n;
11 |         level.resize(n+1);
12 |         ptr.resize(n+1);
13 |         adj.resize(n+1);
14 |         fwd.resize(n+1);
15 |     }
16 |     void addedge(int u, int v, int w){
17 |         fwd[u].push_back({adj[u].size()});
18 |         adj[u].push_back({v, w, adj[v].size()});
19 |         adj[v].push_back({u, 0, adj[u].size() - 1});
20 |     }
21 |     bool bfs(int s, int t){
22 |         fill(level.begin(), level.end(), -1);
23 |         ;
24 |         level[s] = 0;
25 |         queue<int> q;
26 |         q.push(s);
27 |         while(q.size()){
28 |             int x = q.front(); q.pop();
29 |             for(auto edge : adj[x]){
30 |                 if(edge.cap > 0 and level[edge.to] == -1){
31 |                     level[edge.to] = level[x] + 1;
32 |                     q.push(edge.to);
33 |                 }
34 |             }

```

```

35 |         return (level[t] != -1);
36 |     }
37 |     int dfs(int now, int t, int pushed){
38 |         if(now == t) return pushed;
39 |         for(int &i = ptr[now]; i < adj[now].size(); i++){
40 |             auto &w = adj[now][i];
41 |             if(level[now] + 1 != level[w.to] or w.cap == 0) continue;
42 |             int tr = dfs(w.to, t, min(pushed, w.cap));
43 |             if(tr == 0) continue;
44 |             w.cap -= tr;
45 |             adj[w.to][w.rev].cap += tr;
46 |             return tr;
47 |         }
48 |         return 0;
49 |     }
50 |     int compute(){
51 |         int max_flow = 0;
52 |         while(bfs(1,n)){
53 |             fill(ptr.begin(), ptr.end(), 0);
54 |             while(int pushed = dfs(1, n, 1e18)){
55 |                 max_flow += pushed;
56 |             }
57 |         }
58 |         return max_flow;
59 |     }
60 | }
61 | int &flowof(int u, int idx){
62 |     auto &e = adj[u][idx];
63 |     return adj[e.to][e.rev].cap;
64 | }
65 | // flow, path
66 | vector<pair<int, vector<int>>> decompose(int s, int t){
67 |     vector<pair<int, vector<int>>> paths;
68 |     ;
69 |     vector<int> cur(n+1, 0), pos(n+1);
70 |     while(true){
71 |         vector<int> path = {s};
72 |         vector<pair<int,int>> used; //u idx
73 |         fill(pos.begin(), pos.end(), -1);
74 |         ;
75 |         pos[s] = 0;
76 |         int u=s;
77 |         bool cycle = false;
78 |         while(u != t){
79 |             while(cur[u] < fwd[u].size() and flowof(u, fwd[u][cur[u]]) == 0) cur[u]++;
80 |             if(cur[u] == fwd[u].size()) break;
81 |             int idx = fwd[u][cur[u]];
82 |             int v = adj[u][idx].to;
83 |             used.push_back({u, idx});
84 |             if(pos[v] != -1){ cycle = true; u = v; break; }
85 |             pos[v] = path.size();
86 |             path.push_back(v);
87 |             u = v;
88 |         }
89 |         if(used.empty()) break;
90 |         int start = cycle ? pos[u] : 0;

```

```

89     int x = LLONG_MAX;
90     for(int i = start; i < used.size(); i++) x = min(x, flowof(used
    [i].first, used[i].second));
91     for(int i = start; i < used.size(); i++) flowof(used[i].first,
    used[i].second) -= x;
92     if(!cycle) paths.push_back({x, path});
93 }
94 return paths;
95 }
96 };

```

3.3 Edmonds-Karp

```

1 struct max_flow{
2     int n;
3     vector<int> dis, par, mn;
4     vector<vector<int>> g;
5     vector<array<int, 3>> edge;
6     max_flow(int _n){
7         n = _n;
8         dis.resize(n + 5);
9         par.resize(n + 5);
10        mn.resize(n + 5);
11        g.resize(n + 5);
12    }
13    void add_edge(int a, int b, int c){
14        g[a].pb(edge.size());
15        edge.pb({a, b, c});
16        g[b].pb(edge.size());
17        edge.pb({b, a, 0});
18    }
19    int bfs()
20    {
21        for(int i = 1; i <= n; i++) dis[i] = -1;
22        for(int i = 1; i <= n; i++) mn[i] = 1e18;
23        dis[1] = 0;
24        queue<int> q;
25        q.push(1);
26        while(q.size()){
27            int x = q.front(); q.pop();
28            for(int i : g[x]){
29                auto [now, to, w] = edge[i];
30                if(dis[to] == -1 and w > 0){
31                    dis[to] = dis[now] + 1;
32                    par[to] = i;
33                    mn[to] = min(mn[now], w);
34                    q.push(to);
35                }
36            }
37        }
38        if(dis[n] == -1) return 0;
39        int now = n;
40        while(now != 1){
41            auto &[a, b, w] = edge[par[now]];
42            auto &[ra, rb, rw] = edge[par[
    now ^ 1]];

```

```

43        w -= mn[n];
44        rw += mn[n];
45        now = a;
46    }
47    return mn[n];
48 }
49 int max_flow_ans(){
50     int ans = 0;
51     while(true){
52         int t = bfs();
53         if(t == 0) break;
54         ans += t;
55     }
56     return ans;
57 }
58 };

```

3.4 Min Cost Max-Flow

```

1 struct Edge {
2     ll to, cap, cost, rev;
3     Edge() {}
4     Edge(int _to, ll _cap, ll _cost, int
    _rev) : to(_to), cap(_cap), cost(
    _cost), rev(_rev) {}
5 };
6 vector<Edge> adj[N];
7 ll dis[N]; int par[N], par_id[N]; bool
    in_que[N];
8 pair<ll, ll> MCMF(int s, int t) {
9     ll flow = 0, cost = 0;
10    while(true) {
11        memset(dis, 0x3f, sizeof(dis));
12        memset(in_que, 0, sizeof(in_que));
13        queue<int> que;
14        que.push(s); dis[s] = 0;
15        while(!que.empty()) { // SPFA
16            int x = que.front();
17            que.pop(); in_que[x] = 0;
18            for(int i = 0; i < (int)adj[x].
    size(); i++) {
19                Edge &e = adj[x][i];
20                if(e.cap > 0 && dis[e.to] >
    dis[x] + e.cost) {
21                    dis[e.to] = dis[x] + e.
    cost;
22                    par[e.to] = x;
23                    par_id[e.to] = i;
24                    if(!in_que[e.to]) {
25                        in_que[e.to] = 1;
26                        que.push(e.to);
27                    }
28                }
29            }
30        }
31        if(dis[t] >= INF) break;
32        ll mi_flow = INF;
33        for(int i = t; i != s; i = par[i])
34            mi_flow = min(mi_flow, adj[par[i]
    ][par_id[i]].cap);
35        flow += mi_flow, cost += mi_flow *
    dis[t];
36        for(int i = t; i != s; i = par[i]) {

```

```

37        Edge &e = adj[par[i]][par_id[i]
    ];
38        e.cap -= mi_flow;
39        adj[e.to][e.rev].cap += mi_flow;
40    }
41    }
42    return make_pair(flow, cost);
43 }
44 void add_edge(int a, int b, ll cap, ll cost)
45 {
46     adj[a].push_back(Edge(b, cap, cost, (int)
    adj[b].size()));
47     adj[b].push_back(Edge(a, 0, -cost, (int)
    adj[a].size() - 1));
48 }
49 ll hungarian(vector<vector<ll>> cost) {
50     int n = (int)cost.size();
51     for(int i = 0; i < n; i++) {
52         for(int j = 0; j < n; j++) {
53             int L = i, T = j + n;
54             add_edge(L, T, 1, cost[i][j]);
55         }
56     }
57     int s = 2 * n, t = 2 * n + 1;
58     for(int i = 0; i < n; i++) {
59         int L = i, T = i + n;
60         add_edge(s, L, 1, 0);
61         add_edge(T, t, 1, 0);
62     }
63     // MCMF(s, t)
64     for(int i = 0; i < n; i++) {
65         for(Edge &e : adj[i]) {
66             if(e.to == s) continue;
67             int row = i, col = e.to - n;
68             if(e.cap == 0)
69                 // (row + 1) match (col + 1)
70             }
71         }
72     }
73     return MCMF(s, t).second;
74 }

```

3.5 MinCut

```

1 int min_cut(Dinic &d, vector<array<int, 3>>
    &edges){
2     int ans = d.compute_max_flow();
3     int n = d.n;
4     vector<int> vis(n + 1);
5     auto dfs = [&](auto self, int u) -> void
6     {
7         if(vis[u]) return;
8         vis[u] = 1;
9         for(auto &e : d.adj[u])
10            if(e.cap > 0) self(self, e.to);
11    };
12    dfs(dfs, 1);
13    for(auto [u, v, w] : edges){
14        if(w > 0 and vis[u] and !vis[v])
15            cout << u << " " << v << "\n";
16    }
17    return ans;
18 }

```

4 Graph

4.1 bellman-ford

```

1 void bellman_ford(){
2     for(int i = 1; i <= n; i++){
3         bool update = false;
4         for(int j = 1; j <= n; j++){
5             for(auto k : g[j])
6                 if(dis[k.F] > dis[j] + k.S)
7                     update = true, dis[k.F]
    = dis[j] + k.S;
8             if(!update) break;
9             if(i == n) cout << "neg_cycle" << "\n";
10        }
11    }

```

4.2 dijk

```

1 priority_queue<pii, vector<pii>, greater<pii>
    >> q;
2 q.push({0, 1});
3 dis[1] = 0;
4 while(q.size()){
5     auto [d, x] = q.top(); q.pop();
6     if(d > dis[x]) continue;
7     for(auto [to, w] : g[x]){
8         if(dis[to] > dis[x] + w){
9             dis[to] = dis[x] + w;
10            q.push({dis[to], to});
11        }
12    }
13 }

```

4.3 Euler-Path

```

1 void dfs(int x){
2     while(g[x].size()){
3         int nxt = g[x].back();
4         g[x].pop_back();
5         dfs(nxt);
6     }
7     ans.pb(x); // 倒序記錄答案
8 }

```

4.4 FunctionalGraph

```

1 // 0-indexed, 每點恰一條出邊 f[x]
2 // cid[x] : x 所在(或最終到達)的環編號
3 // pos[x] : 環上點在環內的位置, 非環點為 -1
4 // dep[x] : x 走到環需要幾步 (環上為 0)
5 // top[x] : x 第一個走到的環上點
6 // cyc[c] : 第 c 個環的點 (依 f 的順序)

```

```

7 // jump(x, k) : 從 x 走 k 步, O(Log k)
8 struct FunctionalGraph {
9     int n, cnt;
10    vector<int> f, cid, pos, dep, top;
11    vector<vector<int>> cyc, up;
12    FunctionalGraph(int n) : n(n), cnt(0), f
        (n), cid(n, -1),
13        pos(n, -1), dep(n, -1), top(n, -1)
14    {}
15    void addEdge(int u, int v) { f[u] = v; }
16    void work() {
17        vector<int> vis(n, 0); // 0 未訪 1
18        // 在路徑上 2 完成
19        for (int i = 0; i < n; i++) {
20            if (vis[i]) continue;
21            vector<int> path;
22            int x = i;
23            while (!vis[x]) {
24                vis[x] = 1;
25                path.push_back(x);
26                x = f[x];
27            }
28            if (vis[x] == 1) { // 找到新環
29                cyc.emplace_back();
30                int k = find(path.begin(),
31                    path.end(), x) - path.
32                    begin();
33                for (int j = k; j < (int)
34                    path.size(); j++) {
35                    int y = path[j];
36                    pos[y] = j - k;
37                    cid[y] = cnt;
38                    dep[y] = 0;
39                    top[y] = y;
40                    vis[y] = 2;
41                    cyc[cnt].push_back(y);
42                }
43                path.resize(k);
44                cnt++;
45            }
46            for (int j = (int)path.size() -
47                1; j >= 0; j--) {
48                int y = path[j];
49                dep[y] = dep[f[y]] + 1;
50                cid[y] = cid[f[y]];
51                top[y] = top[f[y]];
52            }
53            for (auto y : path) vis[y] = 2;
54        }
55        int LOG = 1;
56        while ((1 << LOG) <= n) LOG++;
57        up.assign(LOG, f);
58        for (int j = 1; j < LOG; j++)
59            for (int i = 0; i < n; i++)
60                up[j][i] = up[j - 1][up[j -
61                    1][i]];
62    }
63    int jump(int x, long long k) {
64        if (k > dep[x]) { // 先走進環, 剩下
65            // 的在環上取模
66            k -= dep[x];
67            x = top[x];
68            auto &c = cyc[cid[x]];
69            return c[(pos[x] + k) % c.size()]
70                ;
71        }
72        for (int j = 0; j < k; j++) k >>= 1
73            if (k & 1) x = up[j][x];
74        return x;
75    }
76    // x 能否走到 y ; 可以的話回傳步數, 否則
77    // -1
78    long long dist(int x, int y) {
79        if (cid[x] != cid[y]) return -1;
80        if (pos[y] == -1) { // y 不在環上:
81            // 必須在 x 往環的路上
82            if (dep[x] < dep[y]) return -1;
83            return jump(x, dep[x] - dep[y])
84                == y ? dep[x] - dep[y] : -1;
85        }
86        int t = top[x], sz = cyc[cid[x]].
87            size();
88        return dep[x] + ((pos[y] - pos[t]) %
89            sz + sz) % sz;
90    }
91 };

```

4.5 SCC

```

1 struct SCC{
2     vector<vector<int>> g, rg;
3     vector<bool> vis, used;
4     vector<int> component, st;
5     int component_cnt = 0, n;
6     void dfs1(int node){
7         vis[node] = true;
8         for(int i : g[node]) if(!vis[i])
9             dfs1(i);
10        st.pb(node);
11    }
12    void dfs2(int node){
13        used[node] = true;
14        for(int i : rg[node]) if(!used[i])
15            dfs2(i);
16        component[node] = component_cnt;
17    }
18    SCC(const vector<vector<int>> v){
19        n = v.size() - 1;
20        g = v;
21        rg.resize(n + 1);
22        vis.resize(n + 1, false);
23        used.resize(n + 1, false);
24        component.resize(n + 1, 0);
25        for(int i = 1; i <= n; i++) for(int
26            j : v[i]) rg[j].pb(i);
27    }
28    vector<int> find_component(){
29        for(int i = 1; i <= n; i++) if(!vis[
30            i]) dfs1(i);
31        reverse(st.begin(), st.end());
32        for(int i : st) if(!used[i]){
33            component_cnt++;
34            dfs2(i);
35        }
36        return component;
37    }
38 };

```

4.6 tarjan scc

```

1 struct tarjan_scc{
2     int n, dfn_cnt = 0, scc_cnt = 0;
3     vector<int> belong, dfn, low, stack;
4     vector<vector<int>> g;
5     void initial(int _n){
6         n = _n;
7         belong.resize(n + 1, 0);
8         dfn.resize(n + 1, 0);
9         low.resize(n + 1, 0);
10        g.resize(n + 1, vector<int>(0));
11    }
12    void addedge(int u, int v){
13        g[u].pb(v);
14    }
15    void tarjan(int u){
16        dfn[u] = low[u] = ++dfn_cnt;
17        stack.pb(u);
18        for(int i : g[u]){
19            if(dfn[i] == 0){
20                tarjan(i);
21                low[u] = min(low[u], low[i]);
22            }
23            else if(belong[i] == 0){
24                low[u] = min(low[u], dfn[i]);
25            }
26        }
27        if(dfn[u] == low[u]){
28            scc_cnt++;
29            int x;
30            do{
31                x = stack.back();
32                belong[x] = scc_cnt;
33                stack.pop_back();
34            }while(x != u);
35        }
36    }
37    void work(){
38        for(int i = 1; i <= n; i++){
39            if(belong[i] == 0){
40                tarjan(i);
41            }
42        }
43    }
44 };

```

4.7 線段樹優化建圖

```

1 #include<bits/stdc++.h>
2 using namespace std;
3 #define KCC ios::sync_with_stdio(0);cin.tie
4 (0);
5 #define pb push_back
6 #define pii pair<long long, long long>
7 #define int long long
8 #define vvi vector<vector<long long>>
9 #define ll long long
10 const int INF = 2e18;

```

```

11 vector<pii> g[(int)5e5 + 5];
12
13 int n, m;
14 void add(int u, int v, int w)
15 {
16     g[u].pb(pii{v, w});
17 }
18 void build(int l, int r, int id)
19 {
20     if(l == r){
21         add(l, n + id, 0);
22         add(n + id, l, 0);
23         return;
24     }
25     int mid = (l + r) >> 1;
26     build(l, mid, id << 1);
27     build(mid + 1, r, id << 1 | 1);
28     add(n + id, n + (id << 1), 0);
29     add(n + id, n + (id << 1 | 1), 0);
30 }
31 // 建立 u -> [ql, qr] 邊權w的有向邊
32 void update(int l, int r, int id, int ql,
33     int qr, int u, int w)
34 {
35     if(ql <= l and r <= qr){
36         add(u, n + id, w);
37         return;
38     }
39     int mid = (l + r) >> 1;
40     if(ql <= mid) update(l, mid, id << 1, ql,
41         qr, u, w);
42     if(qr > mid) update(mid + 1, r, id << 1
43         | 1, ql, qr, u, w);
44 }
45 int dis[(int)5e5 + 5];
46 void Dijk()
47 {
48     priority_queue<pii, vector<pii>, greater
49         <pii>> q;
50     q.push({0, 1});
51     dis[1] = 0;
52     while(q.size()){
53         auto[d, x] = q.top(); q.pop();
54         if(d > dis[x]) continue;
55         for(auto [to, w] : g[x]){
56             if(dis[to] > dis[x] + w){
57                 dis[to] = dis[x] + w;
58                 q.push({dis[to], to});
59             }
60         }
61     }
62 }
63 signed main()
64 {
65     KCC
66     cin >> n >> m;
67     build(1, n, 1);
68     for(int i = 0; i < m; i++){
69         int op; cin >> op;
70         if(op == 1){
71             int u, v, w;
72             cin >> u >> v >> w;
73             g[u].pb(pii{v, w});
74         }
75         else{

```

```

72     int u, l, r, w;
73     cin >> u >> l >> r >> w;
74     update(1, n, 1, l, r, u, w);
75 }
76 }
77 for(int i = 1; i <= 5 * n; i++) dis[i] =
78     1e18;
79 Dijk();
80 for(int i = 1; i <= n; i++) cout << (dis
81 [i] == 1e18 ? -1 : dis[i]) << " ";
82 return 0;
83 }

```

5 Linear Programming

5.1 simplex

```

1  /*target:
2   max \sum_{j=1}^n A_{0,j} * x_j
3   condition:
4   \sum_{j=1}^n A_{i,j} * x_j <= A_{i,0} | i=1~m
5   x_j >= 0 | j=1~n
6   VDB = vector<double*>/
7   template<class VDB>
8   VDB simplex(int m,int n,vector<VDB> a){
9       vector<int> left(m+1), up(n+1);
10      iota(left.begin(), left.end(), n);
11      iota(up.begin(), up.end(), 0);
12      auto pivot = [&](int x, int y){
13          swap(left[x], up[y]);
14          auto k = a[x][y]; a[x][y] = 1;
15          vector<int> pos;
16          for(int j = 0; j <= n; ++j){
17              a[x][j] /= k;
18              if(a[x][j] != 0) pos.push_back(j);
19          }
20          for(int i = 0; i <= m; ++i){
21              if(a[i][y]==0 || i == x) continue;
22              k = a[i][j], a[i][y] = 0;
23              for(int j : pos) a[i][j] -= k*a[x][j];
24          }
25      };
26      for(int x,y;;){
27          for(int i=x+1; i <= m; ++i)
28              if(a[i][0]<a[x][0]) x = i;
29          if(a[x][0]>=0) break;
30          for(int j=y+1; j <= n; ++j)
31              if(a[x][j]<a[x][y]) y = j;
32          if(a[x][y]>=0) return VDB();//infeasible
33          pivot(x, y);
34      }
35      for(int x,y;;){
36          for(int j=y+1; j <= n; ++j)
37              if(a[0][j] > a[0][y]) y = j;
38          if(a[0][y]<=0) break;
39          x = -1;
40          for(int i=1; i<=m; ++i) if(a[i][y] > 0)
41              if(x == -1 || a[i][0]/a[i][y]
42                 < a[x][0]/a[x][y]) x = i;
43          if(x == -1) return VDB();//unbounded
44          pivot(x, y);

```

```

45     }
46     VDB ans(n + 1);
47     for(int i = 1; i <= m; ++i)
48         if(left[i] <= n) ans[left[i]] = a[i][0];
49     ans[0] = -a[0][0];
50     return ans;
51 }

```

6 Number Theory

6.1 basic

```

1  template<typename T>
2  void gcd(const T &a,const T &b,T &d,T &x,T &
3  y){
4      if(!b) d=a,x=1,y=0;
5      else gcd(b,a%b,d,y,x), y-=x*(a/b);
6  }
7  long long int phi[N+1];
8  void phiTable(){
9      for(int i=1;i<=N;i++)phi[i]=i;
10     for(int i=1;i<=N;i++)for(x=i*2;x<=N;x+=i)
11         phi[x]-=phi[i];
12 }
13 void all_divdown(const LL &n) { // all n/x
14     for(LL a=1;a<=n;a=n/(n/(a+1))) {
15         // dosomething;
16     }
17 }
18 const int MAXPRIME = 1000000;
19 int iscom[MAXPRIME], prime[MAXPRIME],
20     primecnt;
21 int phi[MAXPRIME], mu[MAXPRIME];
22 void sieve(void){
23     memset(iscom,0,sizeof(iscom));
24     primecnt = 0;
25     phi[1] = mu[1] = 1;
26     for(int i=2;i<MAXPRIME;++i) {
27         if(!iscom[i]) {
28             prime[primecnt++] = i;
29             mu[i] = -1;
30             phi[i] = i-1;
31         }
32         for(int j=0;j<primecnt;++j) {
33             int k = i * prime[j];
34             if(k>MAXPRIME) break;
35             iscom[k] = prime[j];
36             if(i%prime[j]==0) {
37                 mu[k] = 0;
38                 phi[k] = phi[i] * prime[j];
39                 break;
40             } else {
41                 mu[k] = -mu[i];
42                 phi[k] = phi[i] * (prime[j]-1);
43             }
44         }
45     }
46 }
47 bool g_test(const LL &g, const LL &p, const
48 vector<LL> &v) {

```

```

46     for(int i=0;i<v.size();++i)
47         if(modexp(g,(p-1)/v[i],p)==1)
48             return false;
49     return true;
50 }
51 LL primitive_root(const LL &p) {
52     if(p==2) return 1;
53     vector<LL> v;
54     Factor(p-1,v);
55     v.erase(unique(v.begin(), v.end()), v.end
56             ());
57     for(LL g=2;g<p;++g)
58         if(g_test(g,p,v))
59             return g;
60     puts("primitive_root NOT FOUND");
61     return -1;
62 }
63 int Legendre(const LL &a, const LL &p) {
64     return modexp(a%p,(p-1)/2,p); }
65 LL inv(const LL &a, const LL &n) {
66     LL d,x,y;
67     gcd(a,n,d,x,y);
68     return d==1 ? (x+n)%n : -1;
69 }
70 int inv[maxN];
71 LL invtable(int n,LL P){
72     inv[1]=1;
73     for(int i=2;i<n;++i)
74         inv[i]=(P-(P/i))*inv[P%i]%P;
75 }
76 LL log_mod(const LL &a, const LL &b, const
77 LL &p) {
78     // a ^ x = b ( mod p )
79     int m=sqrt(p+.5), e=1;
80     LL v=inv(modexp(a,m,p), p);
81     map<LL,int> x;
82     x[1]=0;
83     for(int i=1;i<m;++i) {
84         e = LLMul(e,a,p);
85         if(!x.count(e)) x[e] = i;
86     }
87     for(int i=0;i<m;++i) {
88         if(x.count(b)) return i*m + x[b];
89         b = LLMul(b,v,p);
90     }
91     return -1;
92 }
93 LL Tonelli_Shanks(const LL &n, const LL &p)
94 {
95     // x^2 = n ( mod p )
96     if(n==0) return 0;
97     if(Legendre(n,p)!=1) while(1) { puts("SQRT
98         ROOT does not exist"); }
99     int S = 0;
100     LL Q = p-1;
101     while( !(Q&1) ) { Q>>=1; ++S; }
102     if(S==1) return modexp(n%p,(p+1)/4,p);
103     LL z = 2;
104     for(;Legendre(z,p)!=-1;++z)
105         LL c = modexp(z,Q,p);
106         LL R = modexp(n%p,(Q+1)/2,p), t = modexp(n
107             %p,Q,p);

```

```

106     int M = S;
107     while(1) {
108         if(t==1) return R;
109         LL b = modexp(c,1L<<(M-i-1),p);
110         R = LLMul(R,b,p);
111         t = LLMul( LLMul(b,b,p), t, p);
112         c = LLMul(b,b,p);
113         M = i;
114     }
115     return -1;
116 }
117 template<typename T>
118 T Euler(T n){
119     T ans=n;
120     for(T i=2;i*i<=n;++i){
121         if(n%i==0){
122             ans=ans/i*(i-1);
123             while(n%i==0)n/=i;
124         }
125     }
126     if(n>1)ans=ans/n*(n-1);
127     return ans;
128 }
129 //Chinese_remainder_theorem
130 template<typename T>
131 T pow_mod(T n,T k,T m){
132     T ans=1;
133     for(n=(n>m?n%m:n);k;k>>=1){
134         if(k&1)ans=ans*n%m;
135         n=n*n%m;
136     }
137     return ans;
138 }
139 template<typename T>
140 T crt(vector<T> &m,vector<T> &a){
141     T M=1,tM,ans=0;
142     for(int i=0;i<(int)m.size();++i)M*=m[i];
143     for(int i=0;i<(int)a.size();++i){
144         tM=M/m[i];
145         ans=(ans+(a[i]*tM%M)*pow_mod(tM,Euler(m
146             [i])-1,m[i])%M)%M;
147     }
148     /*如果m[i]是質數 · Euler(m[i])-1=m[i]-2 ·
149     就不用算Euler了*/
150     return ans;
151 }
152 //java code
153 //求sqrt(N)的連分數
154 public static void Pell(int n){
155     BigInteger N,p1,p2,q1,q2,a0,a1,a2,g1,g2,h1
156         ,h2,p,q;
157     g1=q2=p1=BigInteger.ZERO;
158     h1=q1=p2=BigInteger.ONE;
159     a0=a1=BigInteger.valueOf((int)Math.sqrt
160         (1.0*n));
161     BigInteger ans=a0.multiply(a0);
162     if(ans.equals(BigInteger.valueOf(n))) {
163         System.out.println("No solution!");
164         return ;
165     }
166     while(true){
167         g2=a1.multiply(h1).subtract(g1);

```

```

167 h2=N.subtract(g2.pow(2)).divide(h1);
168 a2=g2.add(a0).divide(h2);
169 p=a1.multiply(p2).add(p1);
170 q=a1.multiply(q2).add(q1);
171 if(p.pow(2).subtract(N.multiply(q.pow
    (2))).compareTo(BigInteger.ONE)==0)
    break;
172 g1=g2;h1=h2;a1=a2;
173 p1=p2;p2=p;
174 q1=q2;q2=q;
175 }
176 System.out.println(p+" "+q);
177 }

```

6.2 bit set

```

1 void sub_set(int S){
2     int sub=S;
3     do{
4         //對某集合的子集合的處理
5         sub=(sub-1)&S;
6     }while(sub!=S);
7 }
8 void k_sub_set(int k,int n){
9     int comb=(1<<k)-1,S=1<<n;
10    while(comb<S){
11        //對大小為k的子集合的處理
12        int x=comb&-comb,y=comb+x;
13        comb=((comb&~y)/x>>1)|y;
14    }
15 }

```

6.3 cantor expansion

```

1 int factorial[MAXN];
2 void init(){
3     factorial[0]=1;
4     for(int i=1;i<=MAXN;++i)factorial[i]=
        factorial[i-1]*i;
5 }
6 int encode(const vector<int> &s){
7     int n=s.size(),res=0;
8     for(int i=0;i<n;++i){
9         int t=0;
10        for(int j=i+1;j<n;++j)
11            if(s[j]<s[i])++t;
12        res+=t*factorial[n-i-1];
13    }
14    return res;
15 }
16 vector<int> decode(int a,int n){
17     vector<int> res;
18     vector<bool> vis(n,0);
19     for(int i=n-1;i>=0;--i){
20         int t=a/factorial[i],j;
21         for(j=0;j<n;++j)
22             if(!vis[j]){
23                 if(t==0)break;
24                 --t;

```

```

25     }
26     res.push_back(j);
27     vis[j]=1;
28     a%=factorial[i];
29 }
30 return res;
31 }

```

6.4 FFT

```

1 template<typename T,typename VT=vector<
    complex<T> > >
2 struct FFT{
3     const T pi;
4     FFT(const T pi=acos((-1)):pi(pi)){
5         unsigned bit_reverse(unsigned a,int len){
6             a=((a&0x55555555U)<<1)|((a&0xAAAAAAAAU)>>1);
7             a=((a&0x33333333U)<<2)|((a&0xCCCCCCCCU)>>2);
8             a=((a&0x0F0F0F0FU)<<4)|((a&0xFF0F0F0FU)>>4);
9             a=((a&0x00FF00FFU)<<8)|((a&0xFFFF0000U)>>8);
10            a=((a&0x0000FFFFU)<<16)|((a&0xFFFF0000U)>>16);
11            return a>>(32-len);
12        }
13        void fft(bool is_inv,VT &in,VT &out,int N)
14        {
15            int bitlen=__lg(N),num=is_inv?-1:1;
16            for(int i=0;i<N;++i)out[bit_reverse(i,
17                bitlen)]=in[i];
18            for(int step=2;step<=N;step<<=1){
19                const int mh=step>>1;
20                for(int i=0;i<mh;++i){
21                    complex<T> wi=exp(complex<T>(0,i*num
22                        *pi/mh));
23                    for(int j=i;j<N;j+=step){
24                        int k=j+mh;
25                        complex<T> u=out[j],t=wi*out[k];
26                        out[j]=u+t;
27                        out[k]=u-t;
28                    }
29                }
30            }
31            if(is_inv)for(int i=0;i<N;++i)out[i]/=N;
32        }
33    };
34 };

```

6.5 FFT2

```

1 using cd = complex<double>;
2
3 const double PI = acos(-1);
4
5 void fft(vector<cd> &a, bool invert) {
6     int n = a.size();
7
8     // Bit reversal
9     for(int i = 1, j = 0; i < n; i++) {
10         int bit = n >> 1;
11         for(; j & bit; bit >>= 1) j ^= bit;
12

```

```

13         j ^= bit;
14
15         if(i < j) swap(a[i], a[j]);
16     }
17
18     for(int len = 2; len <= n; len <<= 1){
19         double ang = 2 * PI / len * (invert
20             ? -1 : 1);
21         cd wlen(cos(ang), sin(ang));
22
23         for(int i = 0; i < n; i += len){
24             cd w(1);
25
26             for(int j = 0; j < len / 2; j++){
27                 cd u = a[i + j];
28                 cd v = a[i + j + len / 2] *
29                     w;
30
31                 a[i + j] = u + v;
32                 a[i + j + len / 2] = u - v;
33
34                 w *= wlen;
35             }
36         }
37
38         if(invert){
39             for(cd& x : a) x /= n;
40         }
41     }
42     // (a[0] + a[1]x + a[2]x^2 + ...) * (b[0] +
43         b[1]x + b[2]x^2 + ...)
44     vector<long long> multiply(vector<long long>
45         a, vector<long long> b) {
46         int n = 1;
47
48         while(n < a.size() + b.size()) n <<= 1;
49
50         vector<cd> fa(a.begin(), a.end());
51         vector<cd> fb(b.begin(), b.end());
52
53         fa.resize(n);
54         fb.resize(n);
55
56         fft(fa, false);
57         fft(fb, false);
58
59         for(int i = 0; i < n; i++) fa[i] *= fb[i];
60
61         fft(fa, true);
62
63         vector<long long> res(n);
64
65         for(int i = 0; i < n; i++) res[i] =
66             llround(fa[i].real());
67     }
68 }

```

6.6 find real root

```

1 // an*x^n + ... + a1x + a0 = 0;
2 int sign(double x){
3     return x < -eps ? -1 : x > eps;
4 }
5
6 double get(const vector<double>&coef, double
    x){
7     double e = 1, s = 0;
8     for(auto i : coef) s += i*e, e *= x;
9     return s;
10 }
11
12 double find(const vector<double>&coef, int n
    , double lo, double hi){
13     double sign_lo, sign_hi;
14     if( !(sign_lo = sign(get(coef,lo))) )
15         return lo;
16     if( !(sign_hi = sign(get(coef,hi))) )
17         return hi;
18     if(sign_lo * sign_hi > 0) return INF;
19     for(int stp = 0; stp < 100 && hi - lo >
20         eps; ++stp){
21         double m = (lo+hi)/2.0;
22         int sign_mid = sign(get(coef,m));
23         if(!sign_mid) return m;
24         if(sign_lo*sign_mid < 0) hi = m;
25         else lo = m;
26     }
27     return (lo+hi)/2.0;
28 }
29
30 vector<double> cal(vector<double>coef, int n
    ){
31     vector<double>res;
32     if(n == 1){
33         if(sign(coef[1])) res.pb(-coef[0]/coef
34             [1]);
35         return res;
36     }
37     vector<double>dcoef(n);
38     for(int i = 0; i < n; ++i) dcoef[i] = coef
39         [i+1]*(i+1);
40     vector<double>droot = cal(dcoef, n-1);
41     droot.insert(droot.begin(), -INF);
42     droot.pb(INF);
43     for(int i = 0; i1 < droot.size(); ++i){
44         double tmp = find(coef, n, droot[i],
45             droot[i+1]);
46         if(tmp < INF) res.pb(tmp);
47     }
48     return res;
49 }
50
51 int main () {
52     vector<double>ve;
53     vector<double>ans = cal(ve, n);
54     // 視情況把答案 +eps 避免 -0
55 }

```

6.7 FWT

```

1 vector<int> F_OR_T(vector<int> f, bool
  inverse){
2   for(int i=0; (2<<i)<=f.size(); ++i)
3     for(int j=0; j<f.size(); j+=2<<i)
4       for(int k=0; k<(1<<i); ++k)
5         f[j+k+(1<<i)] += f[j+k]*(inverse
          ?-1:1);
6   return f;
7 }
8 vector<int> rev(vector<int> A) {
9   for(int i=0; i<A.size(); i+=2)
10    swap(A[i],A[i^(A.size()-1)]);
11   return A;
12 }
13 vector<int> F_AND_T(vector<int> f, bool
  inverse){
14   return rev(F_OR_T(rev(f), inverse));
15 }
16 vector<int> F_XOR_T(vector<int> f, bool
  inverse){
17   for(int i=0; (2<<i)<=f.size(); ++i)
18     for(int j=0; j<f.size(); j+=2<<i)
19       for(int k=0; k<(1<<i); ++k){
20         int u=f[j+k], v=f[j+k+(1<<i)];
21         f[j+k+(1<<i)] = u-v, f[j+k] = u+v;
22       }
23   if(inverse) for(auto &a:f) a/=f.size();
24   return f;
25 }

```

6.8 LinearCongruence

```

1 pair<LL,LL> LinearCongruence(LL a[],LL b[],
  LL m[],int n) {
2   // a[i]*x = b[i] ( mod m[i] )
3   for(int i=0;i<n;++i) {
4     LL x, y, d = extgcd(a[i],m[i],x,y);
5     if(b[i]%d!=0) return make_pair(-1LL,0LL)
      ;
6     m[i] /= d;
7     b[i] = LLmul(b[i]/d,x,m[i]);
8   }
9   LL lastb = b[0], lastm = m[0];
10  for(int i=1;i<n;++i) {
11    LL x, y, d = extgcd(m[i],lastm,x,y);
12    if((lastb-b[i])%d!=0) return make_pair
      (-1LL,0LL);
13    lastb = LLmul((lastb-b[i])/d,x,(lastm/d)
      )*m[i];
14    lastm = (lastm/d)*m[i];
15    lastb = (lastb+b[i])%lastm;
16  }
17  return make_pair(lastb<0?lastb+lastm:lastb
    ,lastm);
18 }

```

6.9 Lucas

```

1 ll C(ll n, ll m, ll p){ // n!/m!/(n-m)!
2   if(n<m) return 0;

```

```

3   return f[n]*inv(f[m],p)%inv(f[n-m],p)%p;
4 }
5 ll L(ll n, ll m, ll p){
6   if(!m) return 1;
7   return C(n%p,m%p,p)*L(n/p,m/p,p)%p;
8 }
9 ll Wilson(ll n, ll p){ // n!%p
10  if(!n)return 1;
11  ll res=Wilson(n/p, p);
12  if((n/p)%2) return res*(p-f[n%p])%p;
13  return res*f[n%p]%p; //(p-1)!%p=-1
14 }

```

6.10 Matrix

```

1 template<typename T>
2 struct Matrix{
3   using rt = std::vector<T>;
4   using mt = std::vector<rt>;
5   using matrix = Matrix<T>;
6   int r,c;
7   mt m;
8   Matrix(int r,int c):r(r),c(c),m(r,rt(c)){}
9   rt& operator[](int i){return m[i];}
10  matrix operator+(const matrix &a){
11    matrix rev(r,c);
12    for(int i=0;i<r;++i)
13      for(int j=0;j<c;++j)
14        rev[i][j]=m[i][j]+a.m[i][j];
15    return rev;
16  }
17  matrix operator-(const matrix &a){
18    matrix rev(r,c);
19    for(int i=0;i<r;++i)
20      for(int j=0;j<c;++j)
21        rev[i][j]=m[i][j]-a.m[i][j];
22    return rev;
23  }
24  matrix operator*(const matrix &a){
25    matrix rev(r,a.c);
26    matrix tmp(a.c,a.r);
27    for(int i=0;i<a.r;++i)
28      for(int j=0;j<a.c;++j)
29        tmp[j][i]=a.m[i][j];
30    for(int i=0;i<r;++i)
31      for(int j=0;j<a.c;++j)
32        for(int k=0;k<c;++k)
33          rev.m[i][j]+=m[i][k]*tmp[j][k];
34    return rev;
35  }
36  bool inverse(){
37    Matrix t(r,r+c);
38    for(int y=0;y<r;y++){
39      t.m[y][c+y] = 1;
40      for(int x=0;x<c;++x)
41        t.m[y][x]=m[y][x];
42    }
43    if(!t.gas())
44      return false;
45    for(int y=0;y<r;y++){
46      for(int x=0;x<c;++x)
47        m[y][x]=t.m[y][c+x]/t.m[y][y];
48    return true;

```

```

49  }
50  T gas(){
51    vector<T> lazy(r,1);
52    bool sign=false;
53    for(int i=0;i<r;++i){
54      if(m[i][i]==0){
55        int j=i+1;
56        while(j<r&&!m[j][i])j++;
57        if(j==r)continue;
58        m[i].swap(m[j]);
59        sign=!sign;
60      }
61      for(int j=0;j<r;j++){
62        if(i==j)continue;
63        lazy[j]=lazy[j]*m[i][i];
64        T mx=m[j][i];
65        for(int k=0;k<c;k++){
66          m[j][k]=m[j][k]*m[i][i]-m[i][k]*mx
            ;
67        }
68      }
69      T det=sign?-1:1;
70      for(int i=0;i<r;++i){
71        det = det*m[i][i];
72        det = det/lazy[i];
73      }
74      return det;
75    }
76  }
77 };

```

6.11 MillerRobin

```

1 ULL LLMul(ULL a, ULL b, const ULL &mod) {
2   LL ans=0;
3   while(b) {
4     if(b&1) {
5       ans+=a;
6       if(ans>=mod) ans-=mod;
7     }
8     a<<=1, b>>=1;
9     if(a>=mod) a-=mod;
10  }
11  return ans;
12 }
13 ULL mod_mul(ULL a,ULL b,ULL m){
14  a%=m,b%=m; /* fast for m < 2^58 */
15  ULL y=(ULL)((double)a*b/m+0.5);
16  ULL r=(a*b-y*m)%m;
17  return r<0?r+m:r;
18 }
19 template<typename T>
20 T pow(T a,T b,T mod){ //a^b%mod
21  T ans=1;
22  for(;b;a=mod_mul(a,a,mod),b>>=1)
23    if(b&1)ans=mod_mul(ans,a,mod);
24  return ans;
25 }
26 int sprp[3]={2,7,61}; //int範圍可解
27 int llsp[
28  7]={2,325,9375,28178,450775,9780504,
29  1795265022}; //至少 unsigned long long範圍

```

```

29 template<typename T>
30 bool isprime(T n,int *sprp,int num){
31   if(n==2)return 1;
32   if(n<2||n%2==0)return 0;
33   int t=0;
34   T u=n-1;
35   for(;u%2==0;++t)u>>=1;
36   for(int i=0;i<num;++i){
37     T a=sprp[i]%n;
38     if(a==0||a==1||a==n-1)continue;
39     T x=pow(a,u,n);
40     if(x==1||x==n-1)continue;
41     for(int j=0;j<t;j++){
42       x=mod_mul(x,x,n);
43       if(x==1)return 0;
44       if(x==n-1)break;
45     }
46     if(x==n-1)continue;
47     return 0;
48   }
49   return 1;
50 }

```

6.12 NTT

```

1 2615053605667*(2^18)+1,3
2 15*(2^27)+1,31
3 479*(2^21)+1,3
4 7*17*(2^23)+1,3
5 3*3*211*(2^19)+1,5
6 25*(2^22)+1,3
7 template<typename T,typename VT=vector<T> >
8 struct NTT{
9   const T P,G;
10  NTT(T p=(1<<23)*7*17+1,T g=3):P(p),G(g){}
11  unsigned bit_reverse(unsigned a,int len){
12    //Look FFT.cpp
13  }
14  T pow_mod(T n,T k,T m){
15    T ans=1;
16    for(n=(n>=m?n%m:n);k>>=1){
17      if(k&1)ans=ans*n%m;
18      n=n*n%m;
19    }
20    return ans;
21  }
22  void ntt(bool is_inv,VT &in,VT &out,int N)
    {
23    int bitlen=__lg(N);
24    for(int i=0;i<N;++i)out[bit_reverse(i,
      bitlen)]=in[i];
25    for(int step=2,id=1;step<=N;step<=1,++
      id){
26      T wn=pow_mod(G,(P-1)>>id,P),wi=1,u,t;
27      const int mh=step>>1;
28      for(int i=0;i<mh;++i){
29        for(int j=i;j<N;j+=step){
30          u=out[j],t=wi*out[j+mh]%P;
31          out[j]=u+t;
32          out[j+mh]=u-t;
33          if(out[j]>=P)out[j]-=P;
34          if(out[j+mh]<0)out[j+mh]+=P;
35        }

```

```

36         wi=wi*wn%P;
37     }
38 }
39 if(is_invn){
40     for(int i=1;i<N/2;++i)swap(out[i],out[
41         N-i]);
42     T invn=pow_mod(N,P-2,P);
43     for(int i=0;i<N;++i)out[i]=out[i]*invn
44         %P;
45 }
46 }
47 };

```

6.13 Simpson

```

1 double simpson(double a,double b){
2     double c=a+(b-a)/2;
3     return (F(a)+4*F(c)+F(b))*(b-a)/6;
4 }
5 double asr(double a,double b,double eps,
6     double A){
7     double c=a+(b-a)/2;
8     double L=simpson(a,c),R=simpson(c,b);
9     if( abs(L+R-A)<15*eps )
10         return L+R+(L+R-A)/15.0;
11     return asr(a,c,eps/2,L)+asr(c,b,eps/2,R);
12 }
13 double asr(double a,double b,double eps){
14     return asr(a,b,eps,simpson(a,b));
15 }

```

6.14 外星模運算

```

1 //a[0]^(a[1]^a[2]^...)
2 #define maxn 1000000
3 int euler[maxn+5];
4 bool is_prime[maxn+5];
5 void init_euler(){
6     is_prime[1]=1; // 1 不是質數
7     for(int i=1;i<=maxn;i++)euler[i]=i;
8     for(int i=2;i<=maxn;i++){
9         if(!is_prime[i]){ // 是質數
10             euler[i]--;
11             for(int j=i<<1;j<=maxn;j+=i){
12                 is_prime[j]=1;
13                 euler[j]=euler[j]/i*(i-1);
14             }
15         }
16     }
17 }
18 LL pow(LL a,LL b,LL mod){ // a^b%mod
19     LL ans=1;
20     for(;b;a=a*a%mod,b>=1)
21         if(b&1)ans=ans*a%mod;
22     return ans;
23 }
24 bool isless(LL *a,int n,int k){
25     if(*a==1)return k>1;
26     if(--n==0)return *a<k;

```

```

27 int next=0;
28 for(LL b=1;b<k;++next)
29     b*=a;
30 return isless(a+1,n,next);
31 }
32 LL high_pow(LL *a,int n,LL mod){
33     if(*a==1||--n==0)return *a%mod;
34     int k=0,r=euler[mod];
35     for(LL tma=1;tma!=pow(*a,k+r,mod);++k)
36         tma=tma*(a%mod);
37     if(isless(a+1,n,k))return pow(*a,high_pow(
38         a+1,n,k),mod);
39     int tmd=high_pow(a+1,n,r), t=(tmd-k+r)%r;
40     return pow(*a,k+t,mod);
41 }
42 LL a[1000005];
43 int t,mod;
44 int main(){
45     init_euler();
46     scanf("%d",&t);
47     #define n 4
48     while(t--){
49         for(int i=0;i<n;++i)scanf("%Lld",&a[i]);
50         scanf("%d",&mod);
51         printf("%Lld\n",high_pow(a,n,mod));
52     }
53 }

```

6.15 數位統計

```

1 ll d[65], dp[65][2]; // up 區間是不是完整
2 ll dfs(int p,bool is8,bool up){
3     if(!p)return 1; // 回傳0 是不是答案
4     if(!up&&~dp[p][is8])return dp[p][is8];
5     int mx = up?d[p]:9; // 可以用的有那些
6     ll ans=0;
7     for(int i=0;i<=mx;++i){
8         if( is8&&i==7 )continue;
9         ans += dfs(p-1,i==8,up&&i==mx);
10    }
11    if(!up)dp[p][is8]=ans;
12    return ans;
13 }
14 ll f(ll N){
15     int k=0;
16     while(N){ // 把數字先分解到陣列
17         d[++k] = N%10;
18         N/=10;
19     }
20     return dfs(k,false,true);
21 }

```

6.16 質因數分解

```

1 LL func(const LL n,const LL mod,const int c)
2 {
3     return (LLmul(n,n,mod)+c+mod)%mod;
4 }

```

```

4 LL pollorrho(const LL n, const int c) { // 循
5     環長度
6     LL a=1, b=1;
7     a=func(a,n,c)%n;
8     b=func(b,n,c)%n; b=func(b,n,c)%n;
9     while(gcd(abs(a-b),n)!=1) {
10         a=func(a,n,c)%n;
11         b=func(b,n,c)%n; b=func(b,n,c)%n;
12     }
13     return gcd(abs(a-b),n);
14 }
15 void prefactor(LL &n, vector<LL> &v) {
16     for(int i=0;i<12;++i) {
17         while(n%prime[i]==0) {
18             v.push_back(prime[i]);
19             n/=prime[i];
20         }
21     }
22 }
23 void smallfactor(LL n, vector<LL> &v) {
24     if(n<MAXPRIME) {
25         while(isp[is8]) {
26             v.push_back(isp[is8]);
27             n/=isp[is8];
28         }
29         v.push_back(n);
30     } else {
31         for(int i=0;i<primecnt&&prime[i]*prime[i]
32             ]<=n;++i) {
33             while(n%prime[i]==0) {
34                 v.push_back(prime[i]);
35                 n/=prime[i];
36             }
37         }
38         if(n!=1) v.push_back(n);
39     }
40 }
41 void comfactor(const LL &n, vector<LL> &v) {
42     if(n<1e9) {
43         smallfactor(n,v);
44         return;
45     }
46     if(Isprime(n)) {
47         v.push_back(n);
48         return;
49     }
50     LL d;
51     for(int c=3; c<=n; c++) {
52         d = pollorrho(n,c);
53         if(d!=n) break;
54     }
55     comfactor(d,v);
56     comfactor(n/d,v);
57 }
58 void Factor(const LL &x, vector<LL> &v) {
59     LL n = x;
60     if(n==1) { puts("Factor 1"); return; }
61     prefactor(n,v);
62     if(n==1) return;
63     comfactor(n,v);
64 }

```

```

65 sort(v.begin(),v.end());
66 }
67 void AllFactor(const LL &n,vector<LL> &v) {
68     vector<LL> tmp;
69     Factor(n,tmp);
70     v.clear();
71     v.push_back(1);
72     int len;
73     LL now=1;
74     for(int i=0;i<tmp.size();++i) {
75         if(i==0 || tmp[i]!=tmp[i-1]) {
76             len = v.size();
77             now = 1;
78         }
79         now*=tmp[i];
80         for(int j=0;j<len;++j)
81             v.push_back(v[j]*now);
82     }
83 }
84 }
85 }
86 }

```

7 String

7.1 ac 自動機

```

1 int trie[(int)5e5 + 5][26], node_cnt = 0,
2 fail[(int)5e5 + 5]; // ac_automachine
3 int pattern_end[(int)5e5 + 5];
4 void insert(string s, int id)
5 {
6     int u = 0;
7     for(char c : s){
8         int v = c - 'a';
9         if(!trie[u][v]) trie[u][v] = ++
10             node_cnt;
11         u = trie[u][v];
12     }
13     pattern_end[id] = u;
14 }
15 void build_ac_automachine()
16 {
17     //bfs
18     queue<int> q;
19     for(int i = 0; i < 26; i++) if(trie[0][i]
20         ] > 0) q.push(trie[0][i]);
21     while(q.size()){
22         int x = q.front(); q.pop();
23         for(int i = 0; i < 26; i++){
24             if(trie[x][i] > 0){
25                 fail[trie[x][i]] = trie[fail
26                     [x]][i];
27                 q.push(trie[x][i]);
28             }
29             else trie[x][i] = trie[fail[x]][i];
30         }
31     }
32 }

```

7.2 hash

```

1 const int a = 31;
2 const int mod1 = 1e9 + 7;
3 const int mod2 = 1e9 + 9;
4 struct BIT{
5     int n;
6     vector<int> bit1, bit2;
7     BIT(int _n){
8         n = _n;
9         bit1.resize(n + 1, 0);
10        bit2.resize(n + 1, 0);
11    }
12    void add(int x, int val1, int val2){
13        int y = x;
14        for(; x <= n; x += x & -x) bit1[x] =
15            (bit1[x] + val1) % mod1;
16        for(; y <= n; y += y & -y) bit2[y] =
17            (bit2[y] + val2) % mod2;
18    }
19    int que1(int x){
20        int ans = 0;
21        for(; x; x -= x & -x) ans = (ans +
22            bit1[x]) % mod1;
23        return ans;
24    }
25    int que2(int x){
26        int ans = 0;
27        for(; x; x -= x & -x) ans = (ans +
28            bit2[x]) % mod2;
29        return ans;
30    }
31    int query1(int l, int r){
32        return (que1(r) - que1(l - 1) + mod1)
33            % mod1;
34    }
35    int query2(int l, int r){
36        return (que2(r) - que2(l - 1) + mod2)
37            % mod2;
38    }
39    void solve()
40    {
41        string s;
42        cin >> s;
43        int n = s.size();
44        vector<int> f1(n, 1), f2(n, 1);
45        BIT bit(n);
46        for(int i = 1; i < n; i++) f1[i] = (f1[i]
47            - 1) * a) % mod1;
48        for(int i = 1; i < n; i++) f2[i] = (f2[i]
49            - 1) * a) % mod2;
50        for(int i = 0; i < n; i++) bit.add(i +
51            1, f1[i] * (s[i] - 'a' + 1) % mod1,
52            f2[i] * (s[i] - 'a' + 1) % mod2);
53        set<pii> cnt;
54        for(int i = k; i <= n; i++){
55            int sum1 = bit.query1(i - k + 1, i)
56                * fpow(f1[i - k], mod1 - 2, mod1)
57                % mod1;
58            int sum2 = bit.query2(i - k + 1, i)
59                * fpow(f2[i - k], mod2 - 2, mod2)
60                % mod2;
61            cnt.insert({sum1, sum2});
62        }
63    }

```

```

50 cout << cnt.size() << "\n";
51 }

```

7.3 KMP

```

1 vector<int> pi(s.size(), 0);
2 for(int i = 1, j = 0; i < s.size(); i++){
3     while(j > 0 and s[i] != s[j]) j = pi[j] -
4         1;
5     if(s[i] == s[j]) j++;
6     pi[i] = j;

```

7.4 manacher

```

1 string s, t = "^";
2 cin >> s;
3 for(char c : s){
4     t += "#";
5     t += c;
6 }
7 t += "##";
8 vector<int> p(t.size(), 0);
9 int c = 0, r = 0, len = 0, center = 0;
10 for(int i = 1; i < t.size() - 1; i++){
11     if(i < r) p[i] = min(r - i, p[2 * c - i]
12         );
13     while(t[i + p[i] + 1] == t[i - p[i] -
14         1]) p[i]++;
15     if(i + p[i] > r){
16         r = i + p[i];
17         c = i;
18     }
19     if(p[i] > len){
20         len = p[i];
21         center = i;
22     }
23 }
24 cout << s.substr((center - len) / 2, len) <<
25     "\n"; //Longest Palindrome

```

7.5 suffix-array

```

1 vector<int> buildSuffixArray(string s, int n
2 )
3 {
4     vector<int> sa(n), tmp(n), rank(n);
5     /*
6     sa[i]: 從sa[i]開始的後綴字串是第i小
7     rank[i]: 從i開始的後綴字串是第幾小
8     tmp: rank的暫存
9     */
10    for(int i = 0; i < n; i++){ // 初始化長
11        度為1的後綴字串
12        sa[i] = i;
13        rank[i] = s[i] - 'a';

```

```

12 }
13 for(int k = 1; k < n; k <= 1){
14     auto cmp = [&](int i, int j){ //比較
15         長度為2*k的字串
16         if(rank[i] != rank[j]) return
17             rank[i] < rank[j];
18         int ri = i + k < n ? rank[i + k]
19             : -1;
20         int rj = j + k < n ? rank[j + k]
21             : -1;
22         return ri < rj;
23     };
24     sort(sa.begin(), sa.end(), cmp); //
25     sa 按照字典序(rank)排列
26     /*
27     更新rank
28     要先用tmp存，因為cmp會用到rank的舊資
29     料
30     */
31     tmp[sa[0]] = 0;
32     for(int i = 1; i < n; i++) tmp[sa[i]
33         ] = tmp[sa[i - 1]] + (cmp[sa[i]
34             - 1, sa[i]] ? 1 : 0);
35     rank = tmp;
36 }
37 return sa;
38 }
39 vector<int> buildLCP(string s, vector<int>
40     sa, int n)
41 {
42     vector<int> rank(n, 0); // rank[i]: 從s[
43         i]開始的字尾字串按照字典序排序後的位
44         置
45     for(int i = 0; i < n; i++) rank[sa[i]] =
46         i;
47     int h = 0; // 因為是按字串s原始位置順序計
48         算，用前綴相似的特性，h每次最多只會
49         減少1
50     vector<int> lcp(n, 0); // 與字典序前一個
51         的字尾字串的最長共同前綴長度
52     for(int i = 0; i < n; i++){
53         if(rank[i] > 0){
54             int j = sa[rank[i] - 1]; // 字典
55                 序前一個的字尾字串
56             while(i + h < n and j + h < n
57                 and s[i + h] == s[j + h]) h
58                 ++;
59             lcp[rank[i]] = h;
60             if(h > 0) h--;
61         }
62     }
63     return lcp;
64 }
65 int ans = 0, n = s.size();
66 // ans: the number of distinct substrings
67 // that appear in a string
68 for(int i = 0; i < n; i++) ans += n - sa[i]
69     - lcp[i];
70 /*
71 (n - sa[i]): 這個字尾本來可以貢獻這麼多個子
72 字串。
73 lcp[i]: 扣掉跟字典序前一個字尾長得一模一樣的
74 前綴，因為那些已經算過了。

```

53 */

7.6 Zfunction

```

1 vector<int> ZValue(const string &s) {
2     vector<int> z(s.length(), 0);
3     for (int i = 1, l = 1, r = 1; i < (int)
4         s.length(); i++) {
5         if (i < r) {
6             z[i] = min(z[i - 1], r - i);
7         }
8         while (i + z[i] < (int) s.length()
9             && s[i + z[i]] == s[z[i]]) {
10             z[i]++;
11         }
12         if (r < i + z[i]) {
13             l = i;
14             r = i + z[i];
15         }
16     }
17     return z;
18 }

```

8 Tarjan

8.1 dominator tree

```

1 struct dominator_tree{
2     static const int MAXN=5005;
3     int n; // 1-base
4     vector<int> G[MAXN], rG[MAXN];
5     int pa[MAXN], dfn[MAXN], id[MAXN], dfnCn;
6     int semi[MAXN], idom[MAXN], best[MAXN];
7     vector<int> tree[MAXN]; // tree here
8     void init(int _n){
9         n = _n;
10        for(int i=1; i<=n; ++i)
11            G[i].clear(), rG[i].clear();
12    }
13    void add_edge(int u, int v){
14        G[u].push_back(v);
15        rG[v].push_back(u);
16    }
17    void dfs(int u){
18        id[dfn[u]]++dfnCn=u;
19        for(auto v:G[u]) if(!dfn[v])
20            dfs(v),pa[dfn[v]]=dfn[u];
21    }
22    int find(int y,int x){
23        if(y <= x) return y;
24        int tmp = find(pa[y],x);
25        if(semi[best[y]] > semi[best[pa[y]]])
26            best[y] = best[pa[y]];
27        return pa[y] = tmp;
28    }
29    void tarjan(int root){
30        dfnCn = 0;
31        for(int i=1; i<=n; ++i){

```

```

32     dfn[i] = idom[i] = 0;
33     tree[i].clear();
34     best[i] = semi[i] = i;
35 }
36 dfs(root);
37 for(int i=dfnCnt; i>1; --i){
38     int u = id[i];
39     for(auto v:rG[u] if(v=dfn[v]){
40         find(v,i);
41         semi[i]=min(semi[i],semi[best[v]]);
42     }
43     tree[semi[i]].push_back(i);
44     for(auto v:tree[pa[i]]){
45         find(v, pa[i]);
46         idom[v] = semi[best[v]]==pa[i]
47             ? pa[i] : best[v];
48     }
49     tree[pa[i]].clear();
50 }
51 for(int i=2; i<=dfnCnt; ++i){
52     if(idom[i] != semi[i])
53         idom[i] = idom[idom[i]];
54     tree[id[idom[i]]].push_back(id[i]);
55 }
56 }
57 }dom;

```

8.2 EBCC

```

1 struct EBCC { // CF/contest/1986/pF
2     int n, cur, cnt;
3     vector<vector<int>> adj;
4     vector<int> stk, dfn, low, bel;
5     vector<pair<int, int>> bridges; // 關鍵
6     邊
7     EBCC(int n) : n(n), cur
8         (0), cnt(0), adj(n), low(n), dfn(n),
9         -1, bel(n, -1) {}
10    void addEdge(int u, int v) {
11        adj[u].push_back(v);
12        adj[v].push_back(u);
13    }
14    void dfs(int x, int p) {
15        dfn[x] = low[x] = cur++;
16        stk.push_back(x);
17        for (auto y : adj[x]) {
18            if (y == p) continue;
19            if (dfn[y] == -1) {
20                dfs(y, x);
21                low[x] = min(low[x], low[y]);
22            }
23            if (low[y] > dfn[x]) {
24                bridges.emplace_back(x,
25                    y);
26            }
27            else if (bel[y] == -1) {
28                low[x] = min(low[x], dfn[y]);
29            }
30        }
31        if (dfn[x] == low[x]) {
32            int y;

```

```

29    do {
30        y = stk.back();
31        bel[y] = cnt;
32        stk.pop_back();
33    } while (y != x);
34    cnt++;
35 }
36 }
37 vector<int> work() { // not connected
38     for (int i = 0; i < n; i++)
39         if (dfn[i] == -1) dfs(i, -1);
40     return bel;
41 }
42 struct Graph {
43     int n;
44     vector<pair<int, int>> edges;
45     vector<int> siz, cnte;
46 };
47 Graph compress() {
48     Graph g;
49     g.n = cnt;
50     g.siz.resize(cnt);
51     g.cnte.resize(cnt);
52     for (int i = 0; i < n; i++) {
53         g.siz[bel[i]]++;
54         for (auto j : adj[i]) {
55             if (bel[i] < bel[j]) {
56                 g.edges.emplace_back(bel
57                     [i], bel[j]);
58             } else if (bel[i] == bel[j]
59                 && i < j) {
60                 g.cnte[bel[i]]++;
61             }
62         }
63     }
64     return g;
65 }

```

8.3 SCC

```

1 struct SCC {
2     int n, cur, cnt;
3     vector<vector<int>> adj;
4     vector<int> stk, dfn, low, bel;
5     SCC(int n) : n(n), cur(0),
6         cnt(0), adj(n), dfn(n, -1), low(n),
7         bel(n, -1) {}
8     void addEdge(int u, int v) { adj[u].
9         push_back(v); }
10    void dfs(int x) {
11        dfn[x] = low[x] = cur++;
12        stk.push_back(x);
13        for (auto y : adj[x]) {
14            if (dfn[y] == -1) {
15                dfs(y);
16                low[x] = min(low[x], low[y]);
17            }
18            else if (bel[y] == -1) {
19                low[x] = min(low[x], dfn[y]);
20            }
21        }
22    }
23 }

```

```

19     if (dfn[x] == low[x]) {
20         int y;
21         do {
22             y = stk.back();
23             bel[y] = cnt;
24             stk.pop_back();
25         } while (y != x);
26         cnt++;
27     }
28 }
29 vector<int> work() {
30     for (int i = 0; i < n; i++)
31         if (dfn[i] == -1) dfs(i);
32     return bel;
33 }
34 struct Graph {
35     int n;
36     vector<pair<int, int>> edges;
37     vector<int> siz, cnte;
38 };
39 Graph compress() {
40     Graph g;
41     g.n = cnt;
42     g.siz.resize(cnt);
43     g.cnte.resize(cnt);
44     for (int i = 0; i < n; i++) {
45         g.siz[bel[i]]++;
46         for (auto j : adj[i]) {
47             if (bel[i] != bel[j]) {
48                 g.edges.emplace_back(bel
49                     [i], bel[j]);
50             } else {
51                 g.cnte[bel[i]]++;
52             }
53         }
54     }
55     return g;
56 }

```

8.4 tnfsb017 2 sat

```

1 #include<bits/stdc++.h>
2 using namespace std;
3 #define MAXN 8001
4 #define MAXN2 MAXN*4
5 #define n(X) ((X)+2*N)
6 vector<int> v[MAXN2], rv[MAXN2], vis_t;
7 int N,M;
8 void addedge(int s,int e){
9     v[s].push_back(e);
10    rv[e].push_back(s);
11 }
12 int scc[MAXN2];
13 bool vis[MAXN2]={false};
14 void dfs(vector<int> *uv,int n,int k=-1){
15     vis[n]=true;
16     for(int i=0;i<uv[n].size();++i)
17         if(!vis[uv[n][i]])
18             dfs(uv,uv[n][i],k);
19     if(uv==v)vis_t.push_back(n);
20     scc[n]=k;
21 }

```

```

22 void solve(){
23     for(int i=1;i<=N;++i){
24         if(!vis[i])dfs(v,i);
25         if(!vis[n(i)])dfs(v,n(i));
26     }
27     memset(vis,0,sizeof(vis));
28     int c=0;
29     for(int i=vis_t.size()-1;i>=0;--i)
30         if(!vis[vis_t[i]])
31             dfs(rv,vis_t[i],c++);
32 }
33 int main(){
34     int a,b;
35     scanf("%d%d",&N,&M);
36     for(int i=1;i<=N;++i){
37         // (A or B)&(!A & !B) A^B
38         a=i*2-1;
39         b=i*2;
40         addedge(n(a),b);
41         addedge(n(b),a);
42         addedge(a,n(b));
43         addedge(b,n(a));
44     }
45     while(M--){
46         scanf("%d%d",&a,&b);
47         a = a>0?a*2-1:-a*2;
48         b = b>0?b*2-1:-b*2;
49         // A or B
50         addedge(n(a),b);
51         addedge(n(b),a);
52     }
53     solve();
54     bool check=true;
55     for(int i=1;i<=2*N;++i)
56         if(scc[i]==scc[n(i)])
57             check=false;
58     if(check){
59         printf("%d\n",N);
60         for(int i=1;i<=2*N;i+=2){
61             if(scc[i]>scc[i+2*N]) putchar('+');
62             else putchar('-');
63         }
64         puts("");
65     }else puts("0");
66     return 0;
67 }

```

8.5 vbcc

```

1 struct VBCC {
2     int n, cur, cnt;
3     vector<vector<int>> adj, bcc;
4     vector<int> stk, dfn, low;
5     vector<bool> ap;
6     VBCC(int n) : n(n), cur(0), cnt(0), adj(
7         n), bcc(n),
8         dfn(n, -1), low(n), ap(n)
9         {}
10    void addEdge(int u, int v) {
11        adj[u].push_back(v);
12        adj[v].push_back(u);
13    }
14    void dfs(int x, int p) {

```

```

13 dfn[x] = low[x] = cur++;
14 stk.push_back(x);
15 int ch = 0;
16 for (auto y : adj[x]) {
17     if (y == p) continue;
18     if (dfn[y] == -1) {
19         dfs(y, x), ch++;
20         low[x] = min(low[x], low[y]);
21     }
22     if (low[y] >= dfn[x]) {
23         int v;
24         do {
25             v = stk.back();
26             bcc[v].push_back(cnt);
27             stk.pop_back();
28         } while (v != y);
29         bcc[x].push_back(cnt);
30         cnt++;
31     }
32     if (low[y] >= dfn[x] && p != -1) ap[x] = true;
33     low[x] = min(low[x], dfn[y]);
34 }
35 }
36 if (p == -1 && ch > 1) ap[x] = true;
37 }
38 vector<bool> work() {
39     for (int i = 0; i < n; i++)
40         if (dfn[i] == -1) dfs(i, -1);
41     return ap;
42 }
43 };

```

8.6 橋連通分量

```

1 #define N 1005
2 struct edge{
3     int u,v;
4     bool is_bridge;
5     edge(int u=0,int v=0):u(u),v(v),is_bridge(0){}
6 };
7 vector<edge> E;
8 vector<int> G[N]; // 1-base
9 int low[N],vis[N],Time;
10 int bcc_id[N],bridge_cnt,bcc_cnt; // 1-base
11 int st[N],top; // BCC用
12 void add_edge(int u,int v){
13     G[u].push_back(E.size());
14     E.emplace_back(u,v);
15     G[v].push_back(E.size());
16     E.emplace_back(v,u);
17 }
18 void dfs(int u,int re=-1){ // u當前點, re為u連
19     接前一個點的邊
20     int v;
21     low[u]=vis[u]=++Time;
22     st[top++]=u;
23     for(int e:G[u]){

```

```

23         v=E[e].v;
24         if(!vis[v]){
25             dfs(v,e^1); // e^1 反向邊
26             low[u]=min(low[u],low[v]);
27             if(vis[u]<low[v]){
28                 E[e].is_bridge=E[e^1].is_bridge=1;
29                 ++bridge_cnt;
30             }
31             }else if(vis[v]<vis[u]&&e!=re){
32                 low[u]=min(low[u],vis[v]);
33             }
34         if(vis[u]==low[u]){ // 處理BCC
35             ++bcc_cnt; // 1-base
36             do bcc_id[v=st[--top]]=bcc_cnt; // 每個點
37                 所在的BCC
38             while(v!=u);
39         }
40     void bcc_init(int n){
41         Time=bcc_cnt=bridge_cnt=top=0;
42         E.clear();
43         for(int i=1;i<=n;++i){
44             G[i].clear();
45             vis[i]=bcc_id[i]=0;
46         }
47     }

```

8.7 雙連通分量 & 割點

```

1 #define N 1005
2 vector<int> G[N]; // 1-base
3 vector<int> bcc[N]; // 存每塊雙連通分量的點
4 int low[N],vis[N],Time;
5 int bcc_id[N],bcc_cnt; // 1-base
6 bool is_cut[N]; // 是否為割點
7 int st[N],top;
8 void dfs(int u,int pa=-1){ // u當前點, pa父親
9     int t, child=0;
10    low[u]=vis[u]=++Time;
11    st[top++]=u;
12    for(int v:G[u]){
13        if(!vis[v]){
14            dfs(v,u),++child;
15            low[u]=min(low[u],low[v]);
16            if(vis[u]<=low[v]){
17                is_cut[u]=1;
18                bcc[++bcc_cnt].clear();
19                do{
20                    bcc_id[t=st[--top]]=bcc_cnt;
21                    bcc[bcc_cnt].push_back(t);
22                }while(t!=v);
23                bcc_id[u]=bcc_cnt;
24                bcc[bcc_cnt].push_back(u);
25            }
26        }else if(vis[v]<vis[u]&&v!=pa){ // 反向邊
27            low[u] = min(low[u],vis[v]);
28        } // u是dfs樹的根要特判
29        if(pa!=-1&&child<2)is_cut[u]=0;
30    }
31    void bcc_init(int n){
32        Time=bcc_cnt=top=0;

```

```

33     for(int i=1;i<=n;++i){
34         G[i].clear();
35         is_cut[i]=vis[i]=bcc_id[i]=0;
36     }
37 }

```

9 Tree Problem

9.1 centroid decomposition

1 Given a tree of n nodes, your task is to count the number of distinct paths that have at least k₁ and at most k₂ edges.

2 Input

3 The first input line contains three integers n, k₁ and k₂: the number of nodes and the path lengths. The nodes are numbered 1,2,...,n.

4 Then there are n-1 lines describing the edges. Each line contains two integers a and b: there is an edge between nodes a and b.

5 Output

6 Print one integer: the number of paths.

7 Constraints

8

9 1 ≤ k₁ ≤ k₂ ≤ n ≤ 2 × 10⁵

10 1 ≤ a, b ≤ n

11

12 Example

13 Input:

14 5 2 3

15 1 2

16 2 3

17 3 4

18 3 5

19

20 Output:

21 6

22 #include<bits/stdc++.h>

23 using namespace std;

24 #pragma GCC optimize("Ofast")

25 #define KCC ios::sync_with_stdio(0);cin.tie(0);

26 #define pb push_back

27 #define pii pair<long long, long long>

28 #define F first

29 #define S second

30 // #define int long long

31 #define MAXN (int)2e5 + 5

32

33 struct BIT{

34 int n;

35 vector<long long> bit;

36 void initial(int _n){

37 n = _n;

38 bit.resize(n + 5, 0);

39 }

40 void upd(int x, long long val){

41 x++;

```

42         for(; x <= n; x += x & -x) bit[x] += val;
43     }
44     long long query(int x){
45         x++;
46         long long ans = 0;
47         for(; x > 0; x -= x & -x) ans += bit[x];
48         return ans;
49     }
50 }bit;
51
52 int n, k1, k2;
53 vector<int> g[MAXN];
54 bool removed[MAXN];
55 int sz[MAXN];
56 long long ans = 0;
57 void dfs_sz(int x, int p)
58 {
59     sz[x] = 1;
60     for(int i : g[x]){
61         if(i == p or removed[i]) continue;
62         dfs_sz(i, x);
63         sz[x] += sz[i];
64     }
65 }
66 int dfs_centroid(int x, int p, int tot)
67 {
68     for(int i : g[x]){
69         if(i == p or removed[i]) continue;
70         if(sz[i] > (tot >> 1)) return dfs_centroid(i, x, tot);
71     }
72     return x;
73 }
74 void dfs_dis(int x, int p, int d, vector<int> &dis)
75 {
76     if(d > k2) return;
77     dis.pb(d);
78     for(int i : g[x]){
79         if(i == p or removed[i]) continue;
80         dfs_dis(i, x, d + 1, dis);
81     }
82 }
83 vector<int> dis, used, cnt(MAXN, 0);
84 void decompose(int x)
85 {
86     dfs_sz(x, -1);
87     int c = dfs_centroid(x, -1, sz[x]);
88     removed[c] = true;
89     used.clear();
90     bit.upd(0, 1);
91     cnt[0]++;
92     used.pb(0);
93     for(int i : g[c]){
94         if(removed[i]) continue;
95         dis.clear();
96         dfs_dis(i, c, 1, dis);
97
98         for(int d : dis){
99             if(k2 >= d) ans += bit.query(k2 - d) - bit.query(k1 - d - 1);
100         }
101     }

```

```

102     k2 >= d + x >= k1
103     k2 - d >= x >= k1 - d
104     */
105     for(int d : dis){
106         bit.upd(d, 1);
107         cnt[d]++;
108         if(cnt[d] == 1) used.pb(d);
109     }
110 }
111 for(int i : used) bit.upd(i, -cnt[i]);
112 for(int i : used) cnt[i] = 0;
113
114 for(int i : g[c]){
115     if(!removed[i]){
116         decompose(i);
117     }
118 }
119 }
120 signed main()
121 {
122     KCC
123     cin >> n >> k1 >> k2;
124     for(int i = 0; i < n - 1; i++){
125         int u, v; cin >> u >> v;
126         g[u].pb(v);
127         g[v].pb(u);
128     }
129     // cnt.assign(k + 1, 0);
130     bit.initial(k2 + 1);
131     decompose(1);
132     cout << ans << "\n";
133     return 0;
134 }

```

9.2 find-centroid

```

1 #include<bits/stdc++.h>
2 using namespace std;
3 #define int long long
4 #define KCC ios::sync_with_stdio(0);cin.tie
5 (0);
6 #define pb push_back
7
8 int subtree[(int)2e5 + 5], n;
9 vector<int> g[(int)2e5 + 5];
10 void dfs(int now, int par)
11 {
12     subtree[now] = 1;
13     for(auto w : g[now]){
14         if(w != par){
15             dfs(w, now);
16             subtree[now] += subtree[w];
17         }
18     }
19     return ;
20 }
21 int get(int now, int par)
22 {
23     for(auto w : g[now]){
24         if(w != par and subtree[w] * 2 > n)
25             return get(w, now);
26     }
27     return now;
28 }

```

```

26 }
27 signed main()
28 {
29     KCC
30     cin >> n;
31     for(int i = 0; i < n - 1; i++){
32         int a, b; cin >> a >> b;
33         g[a].pb(b);
34         g[b].pb(a);
35     }
36     dfs(1, 0);
37     cout << get(1, 0) << "\n";
38     return 0;
39 }

```

9.3 hld

```

1 vector<int> edge[MAXN];
2 int ar[MAXN], sz[MAXN], par[MAXN], son[MAXN
3 ], dep[MAXN], dfn[MAXN], top[MAXN], t =
4 0;
5 void dfs_son(int x, int p, int d)
6 {
7     sz[x] = 1;
8     dep[x] = d;
9     son[x] = 0;
10    for(int i : edge[x]){
11        if(i != p){
12            dfs_son(i, x, d + 1);
13            sz[x] += sz[i];
14            par[i] = x;
15            if(sz[i] > sz[son[x]]) son[x] =
16                i;
17        }
18    }
19 }
20 void dfs_hld(int x, int p)
21 {
22     dfn[x] = ++t;
23     if(son[x] != 0){
24         top[son[x]] = top[x];
25         dfs_hld(son[x], x);
26     }
27     for(int i : edge[x]){
28         if(i != p and i != son[x]){
29             top[i] = i;
30             dfs_hld(i, x);
31         }
32     }
33 }
34 int main()
35 {
36     int n, q;
37     cin >> n >> q;
38     for(int i = 1; i <= n; i++) cin >> ar[i];
39     for(int i = 0; i < n - 1; i++){
40         int u, v;
41         cin >> u >> v;
42         edge[u].pb(v);
43         edge[v].pb(u);
44     }
45     dfs_son(1, 0, 1);
46 }

```

```

43 top[1] = 1;
44 dfs_hld(1, 0);
45
46 vector<int> v(n + 5);
47 for(int i = 1; i <= n; i++) v[dfn[i]] =
48     ar[i];
49 seg_tree seg(n);
50 seg.build(v, 1, n, 1);
51 while(q--){
52     int op; cin >> op;
53     if(op == 1){ //change the value of
54         node s to x
55         int s, x;
56         cin >> s >> x;
57         seg.upd(dfn[s], x, 1, n, 1);
58     }
59     else{ //find the maximum value on
60         the path between nodes a and b.
61         int a, b, ans = 0;
62         cin >> a >> b;
63         while(top[a] != top[b]){
64             if(dfn[top[a]] < dfn[top[b]]
65                 swap(a, b);
66             ans = max(ans, seg.query(dfn
67                 [top[a]], dfn[a], 1, n,
68                 1));
69             a = par[top[a]];
70         }
71         if(dfn[a] > dfn[b]) swap(a, b);
72         ans = max(ans, seg.query(dfn[a],
73             dfn[b], 1, n, 1));
74         cout << ans << " ";
75     }
76 }

```

9.4 lca

```

1 void dfs(int now, int par)
2 {
3     deep[now] = deep[par] + 1;
4     p[now][0] = par;
5     for(int i : v[now]) if(i != par) dfs(i,
6         now);
7     return;
8 }
9 int lca(int u, int v)
10 {
11     int x = deep[u], y = deep[v];
12     if(x > y){
13         swap(u, v);
14     }
15     y -= x; // x < y
16     for(int i = 25; i >= 0; i--) if(y & (1 <<
17         i)) v = p[v][i];
18     if(u == v) return u;
19     for(int i = 25; i >= 0; i--){
20         if(p[u][i] != p[v][i]){
21             u = p[u][i];
22             v = p[v][i];
23         }
24     }
25 }

```

```

24     return p[u][0];
25 }
26 for(int j = 1; j < 30; j++){
27     for(int i = 2; i <= n; i++){
28         p[i][j] = p[p[i][j - 1]][j - 1];
29     }
30 }

```

9.5 tree-hash

```

1 #include<bits/stdc++.h>
2 using namespace std;
3 #define KCC ios::sync_with_stdio(0);cin.tie
4 (0);
5 #define pb push_back
6 #define pii pair<long long, long long>
7 #define F first
8 #define S second
9 #define int long long
10 #define ll long long
11 map<vector<int>, int> id;
12 int cnt = 0;
13 int get_id(vector<int> v)
14 {
15     if(!id.count(v)) id[v] = ++cnt;
16     return id[v];
17 }
18 int dfs(int now, int par, vector<vector<int>
19     >> &g)
20 {
21     vector<int> v;
22     for(int i : g[now]) if(i != par){
23         v.pb(dfs(i, now, g));
24     }
25     sort(v.begin(), v.end());
26     return get_id(v);
27 }
28 vector<int> get_center(const vector<vector<
29     int>> &g)
30 {
31     int n = g.size() - 1;
32     vector<int> deg(n + 1, 0);
33     queue<int> q;
34     for(int i = 1; i <= n; i++){
35         deg[i] = g[i].size();
36         if(deg[i] == 1) q.push(i);
37     }
38     int remain = n;
39     while(remain > 2){
40         int t = q.size();
41         remain -= t;
42         while(t--){
43             int x = q.front(); q.pop();
44             for(int i : g[x]){
45                 deg[i]--;
46                 if(deg[i] == 1) q.push(i);
47             }
48         }
49     }
50     vector<int> center;
51     while(q.size()){
52         int x = q.front(); q.pop();
53     }

```

```

51     center.pb(x);
52 }
53 return center;
54 }
55 void solve()
56 {
57     int n;
58     cin >> n;
59     vector<vector<int>> tree1, tree2;
60     tree1.resize(n + 1);
61     tree2.resize(n + 1);
62
63     for(int i = 0; i < n - 1; i++){
64         int u, v;
65         cin >> u >> v;
66         tree1[u].pb(v);
67         tree1[v].pb(u);
68     }
69     for(int i = 0; i < n - 1; i++){
70         int u, v;
71         cin >> u >> v;
72         tree2[u].pb(v);
73         tree2[v].pb(u);
74     }
75     vector<int> center1 = get_center(tree1);
76     vector<int> center2 = get_center(tree2);
77     int h1 = min(dfs(center1[0], 0, tree1),
78                 dfs(center1.back(), 0, tree1));
79     int h2 = min(dfs(center2[0], 0, tree2),
80                 dfs(center2.back(), 0, tree2));
81
82     if(h1 != h2) cout << "NO\n";
83     else cout << "YES\n";
84 }

```

10 compare

10.1 check

```

1 #include<bits/stdc++.h>
2 using namespace std;
3 int main() {
4     // 1. 在迴圈外編譯，並檢查是否成功
5     cout << "Compiling files..." << endl;
6
7     // system() 回傳 0 代表成功執行
8     int c1 = system("g++ -Wall -std=c++20
9     sol.cpp -o sol.exe");
10    int c2 = system("g++ -Wall -std=c++20 bf
11    .cpp -o bf.exe");
12    int c3 = system("g++ -Wall -std=c++20
13    gen.cpp -o gen.exe");
14
15    if (c1 != 0 || c2 != 0 || c3 != 0) {
16        cout << "Compilation Failed! Check
17        if sol.cpp, bf.cpp, and gen.cpp
18        exist." << endl;
19        return 1;
20    }
21 }

```

```

17 cout << "Compilation successful.
18 Starting stress test..." << endl;
19
20 for (int T = 1; ; T++) {
21     // 2. 執行程序
22     system("gen.exe > input.txt");
23     system("sol.exe < input.txt > sol.
24     out");
25     system("bf.exe < input.txt > bf.out"
26     );
27
28     // 3. 使用 fc /W 比對
29     if (system("fc sol.out bf.out > nul"
30     )) {
31         system("fc sol.out bf.out");
32         cout << "WA on Test #" << T << "
33         !" << endl;
34         // 發生錯誤時停止，此時 input.
35         txt 就是出錯的測資
36         break;
37     } else {
38         if (T % 10 == 0) cout << "Passed
39         " << T << " tests..." <<
40         endl;
41     }
42 }
43
44 return 0;
45 }

```

10.2 gen

```

1 #include<bits/stdc++.h>
2 using namespace std;
3
4 // 使用隨機種子，確保每次生成的數據都不同
5 mt19937_64 rng(chrono::steady_clock::now().
6 time_since_epoch().count());
7
8 // 生成 [L, r] 範圍內的隨機整數
9 long long rnd(long long l, long long r) {
10     return uniform_int_distribution<long
11     long>(l, r)(rng);
12 }
13
14 // 生成一棵隨機樹 (N 個點，N-1 條邊)
15 void gen_tree(int n) {
16     for (int i = 2; i <= n; i++) {
17         // 讓每個點 i 連向 [1, i-1] 中的隨機
18         點
19         int u = i;
20         int v = rnd(1, i - 1);
21         cout << u << " " << v << "\n";
22     }
23 }

```

11 default

11.1 debug

```

1 #ifdef DEBUG
2 #define dbg(...) {\
3     fprintf(stderr, "%s - %d : (%s) = ",
4     __PRETTY_FUNCTION__, __LINE__, #
5     __VA_ARGS__); \
6     _DO(__VA_ARGS__); \
7 }
8 template<typename I> void _DO(I&&x){cerr<<x
9 <<endl;}
10 template<typename I, typename...T> void _DO(I
11 &&x, T&&...tail){cerr<<x<<" "; _DO(tail
12 ...);}
13 #else
14 #define dbg(...)
15 #endif

```

11.2 ext

```

1 #include<bits/extc++.h>
2 #include<ext/pd_ds/assoc_container.hpp>
3 #include<ext/pd_ds/tree_policy.hpp>
4 using namespace __gnu_cxx;
5 using namespace __gnu_pbds;
6 template<typename T>
7 using pbds_set = tree<T, null_type, less<T>,
8 rb_tree_tag,
9 tree_order_statistics_node_update>;
10 template<typename T, typename U>
11 using pbds_map = tree<T, U, less<T>,
12 rb_tree_tag,
13 tree_order_statistics_node_update>;
14 using heap = __gnu_pbds::priority_queue<int>;
15 //s.find_by_order(1); //0 base
16 //s.order_of_key(1);

```

11.3 IncStack

```

1 //Magic
2 #pragma GCC optimize "Ofast"
3 //stack resize, change esp to rsp if 64-bit
4 system
5 asm("mov %0, %%esp\n" :: "g"(mem+10000000));
6 -Wl,--stack,214748364 -trigraphs
7 #pragma comment(linker, "/STACK
8 :1024000000,1024000000")
9 //linux stack resize
10 #include<sys/resource.h>
11 void increase_stack(){
12     const rlim_t ks=64*1024*1024;
13     struct rlimit rl;
14     int res=getrlimit(RLIMIT_STACK,&rl);
15     if(!res&&rl.rlim_cur<ks){
16         rl.rlim_cur=ks;
17     }
18 }

```

```

15 res=setrlimit(RLIMIT_STACK,&rl);
16 }
17 }

```

11.4 input

```

1 inline int read(){
2     int x=0; bool f=0; char c=getchar();
3     while(ch<'0' || '9'<ch)f|=ch=='-',ch=getchar
4     ();
5     while('0'<=ch&&ch<='9')x=x*10-'0'+ch,ch=
6     getchar();
7     return f?-x:x;
8 }
9 // #!/bin/bash
10 // g++ -std=c++11 -O2 -Wall -Wextra -Wno-
11 unused-result -DDEBUG $1 && ./a.out
12 // -fsanitize=address -fsanitize=undefined
13 -fsanitize=return

```

12 dp 優化

12.1 2D1D 優化

```

1 /*
2 dp(L, r) = min L≤k<r {dp(L, k) + dp(k + 1, r)
3 + w(L, r)} L ≤ r
4 dp(L, r) = min L≤k<r {dp(L, k) + w(k + 1, r)}
5 L ≤ r
6 以上暴力會會是 O(n^3) · 若 w 滿足四邊形不等式
7 以及區間單調
8 性，總體時間複雜度可壓到 O(n^2) ·
9 區間單調性
10 若 a, b, c, d, a ≤ b ≤ c ≤ d, w(a, d) ≥ w(b, c)
11 )
12 若 w 滿足四邊形不等式以及區間單調性，那麼這
13 個 DP 也
14 滿足四邊形不等式。
15 如果 2D/1D DP 滿足四邊形不等式，令 p(L, r) 代
16 表 dp(L, r)
17 的最優決策點，則有 p(L, r - 1) ≤ p(L, r) ≤ p(L
18 + 1, r)
19
20 vector<vector<ll>> dp(n, vector<ll>(n, inf));
21 ;
22 vector<vector<int>> p(n, vector<int>(n));
23 ll w(int l, int r) {} // [L, r]
24 for (int len = 0; len < n; len++) {
25     for (int i = 0; i + len < n; i++) {
26         int j = i + len;
27         if (i == j) {
28             dp[i][j] = 0, p[i][j] = i;
29             continue;
30         }
31     }
32 }

```

```

25     }
26     for (int k = p[i][j - 1]; k <= min(p
27         [i + 1][j], j - 1); k++) {
28         ll x = dp[i][k] + dp[k + 1][j] +
29             w(i, j);
30         if (x < dp[i][j]) dp[i][j] = x,
31             p[i][j] = k;

```

12.2 CHT

```

1 // 當放入的直線斜率單調且查詢單調可以使用
2 // deque
3 // 否則用李超樹
4 struct Line
5 {
6     int m, b;
7     int val(int x){
8         return m * x + b;
9     };
10 bool bad(Line a, Line b, Line c)
11 {
12     // ab交點x座標 >= bc交點x座標
13     // (b.b - a.b) / (a.m - b.m) >= (c.b - b
14     .b) / (b.m - c.m)
15     return (b.b - a.b) * (b.m - c.m) >= (c.b
16     - b.b) * (a.m - b.m);
17 }
18 deque<Line> dq;
19 dq.pb({x, 0});
20 for(int i = 1; i <= n; i++){
21     while(dq.size() > 1 and dq[0].val(s[i])
22         >= dq[1].val(s[i])) dq.pop_front();
23     dp[i] = dq[0].val(s[i]);
24     Line tmp = {f[i], dp[i]};
25     while(dq.size() >= 2 and bad(dq[dq.size
26     () - 2], dq.back(), tmp)) dq.
27     pop_back();
28     dq.pb(tmp);
29 }

```

12.3 SOS

```

1 // 使用情況：跟 bit 與(被)包含有關，且 x 在
2 // 1E6 左右
3 // 題目：一數組，問有多少所有數 & 起來為 0
4 // 的集合數
5 // dp[
6 // x] 代表包含 x 的 y 個數(比 x 大且 bit
7 // 1 全包含 x 的有幾個)
8 // 答案應該包含在 dp[0] 內，但是有重複元素，
9 // 所以考慮容斥
10 // => ans = \sum_{i=0}^n (-1)^{pop_count(
11 // i)} 2^{dp[i]-1}
12 // => 全

```

```

8 // 部為 0 的個數 - 至少一個為 1 的個數 +
9 // 至少兩個為 1 的個數
10 void solve() {
11     int n; cin >> n; Z ans = 0;
12     vector<int> a(n);
13     for (int i = 0; i < n; i++) cin >> a[i];
14     int m = __lg(*max_element(a.begin(), a.
15     end())) + 1;
16     // 定義 dp[mask] 為 mask 被包含於 a[i]
17     // 的 i 個數
18     vector<ll> dp(1 << m);
19     for (int i = 0; i < n; i++) dp[a[i]]++;
20     for (int i = 0; i < m; i++) {
21         for (int mask = 0; mask < 1 << m;
22             mask++) {
23             if (mask >> i & 1) {
24                 int pre = mask ^ (1 << i);
25                 dp[pre] += dp[mask];
26             }
27         }
28     }
29     for (int mask = 0; mask < 1 << m; mask
30         ++){
31         int sgn = __builtin_popcount(mask) &
32             1 ? -1 : 1;
33         ans += sgn * (power(Z(2), dp[mask])
34             - 1);
35     }
36 }
37 // x | y = x, 代表包含於 x 的 y 個數，定義為
38 // dp[x][0]
39 // x & y = x, 代表包含 x 的 y 個數，定義為
40 // dp[x][1]
41 // x & y != 0, 代表至少有一個位元都為 1 的 y
42 // 個數，n - dp[~x][0]
43 void solve() {
44     int n; cin >> n;
45     vector<int> a(n);
46     map<int, int> mp;
47     for (int i = 0; i < n; i++) {
48         cin >> a[i]; mp[a[i]]++;
49     }
50     int m = __lg(*max_element(a.begin(), a.
51     end())) + 1;
52     vector<array<ll, 2>> dp(1 << m);
53     for (int i = 0; i < n; i++) dp[a[i]
54     ][0]++, dp[a[i]][1]++;
55     for (int i = 0; i < m; i++) {
56         for (int mask = 0; mask < 1 << m;
57             mask++) {
58             if (mask >> i & 1) {
59                 int pre = mask ^ (1 << i);
60                 dp[mask][0] += dp[pre][0];
61                 dp[pre][1] += dp[mask][1];
62             }
63         }
64     }
65     for (int i = 0; i < n; i++) {
66         cout << dp[a[i]][0] << " " << dp[a[i]
67         ][1] <<
68         " " << n - (dp[((1 << m) - 1) ^
69         a[i]][0]) << "\n";
70     }
71 }

```

12.4 分治優化

```

1 /*
2 dp(i) = h(i) + min/max j<i {dp(j) + w(j, i)}
3 其中 w(j, i) 必須滿足四邊形不等式。
4 四邊形不等式
5 對於任意 a ≤ b ≤ c ≤ d，若滿足
6 w(a, c) + w(b, d) ≤ (≥) w(a, d) + w(b, c)
7 則稱 w 滿足四邊形不等式。
8 */
9 ll w(int l, int r) {} // (l, r]
10 void solve(int j, int l, int r, int optl,
11     int optr) {
12     if (l > r) return;
13     int m = (l + r) / 2;
14     dp[j][m] = INF;
15     int optm = -1;
16     for (int p = optl; p <= min(m - 1, optr)
17         ; p++) {
18         ll val = dp[j - 1][p] + w(p, m);
19         if (val < dp[j][m]) {
20             dp[j][m] = val;
21             optm = p;
22         }
23     }
24     solve(j, l, m - 1, optl, optm);
25     solve(j, m + 1, r, optm, optr);
26 }

```

13 other

13.1 WhatDay

```

1 int whatday(int y, int m, int d){
2     if(m<=2)m+=12,--y;
3     if(y<1752||y==1752&&m<9||y==1752&&m==9&&d
4         <3)
5         return (d+2*m+3*(m+1)/5+y+y/4+y/5)%7;
6     return (d+2*m+3*(m+1)/5+y+y/4-y/100+y/400)
7         %7;
8 }

```

13.2 上下最大正方形

```

1 void solve(int n, int a[], int b[]) { // 1-base
2     int ans = 0;
3     deque<int> da, db;
4     for (int l = 1, r = 1; r <= n; ++r) {
5         while (da.size() && a[da.back()] >= a[r]) {
6             da.pop_back();
7         }
8         da.push_back(r);
9         while (db.size() && b[db.back()] >= b[r]) {
10             db.pop_back();
11         }
12         db.push_back(r);

```

```

13     for (int d = a[da.front()] + b[db.front()]; r-
14         l+1 > d; ++l) {
15         if (da.front() == 1) da.pop_front();
16         if (db.front() == 1) db.pop_front();
17         if (da.size() && db.size()) {
18             d = a[da.front()] + b[db.front()];
19         }
20     }
21     ans = max(ans, r - l + 1);
22 }
23 printf("%d\n", ans);

```

13.3 最大矩形

```

1 LL max_rectangle(vector<int> s){
2     stack<pair<int, int> > st;
3     st.push(make_pair(-1, 0));
4     s.push_back(0);
5     LL ans = 0;
6     for (size_t i = 0; i < s.size(); ++i) {
7         int h = s[i];
8         pair<int, int> now = make_pair(h, i);
9         while (h < st.top().first) {
10             now = st.top();
11             st.pop();
12             ans = max(ans, (LL)(i - now.second) * now.
13                 first);
14         }
15         if (h > st.top().first) {
16             st.push(make_pair(h, i));
17         }
18     }
19     return ans;

```

14 other language

14.1 java

14.1.1 文件操作

```

1 import java.io.*;
2 import java.util.*;
3 import java.math.*;
4 import java.text.*;
5
6 public class Main{
7
8     public static void main(String args[]){
9         throws FileNotFoundException,
10             IOException
11         Scanner sc = new Scanner(new FileReader(
12             "a.in"));
13         PrintWriter pw = new PrintWriter(new
14             FileWriter("a.out"));
15         int n, m;

```

```

12 n=sc.nextInt();//读入下一个INT
13 m=sc.nextInt();
14
15 for(ci=1; ci<=c; ++ci){
16     pw.println("Case #"+ci+": easy for
17         output");
18 }
19 pw.close();//关闭流并释放 · 这个很重要 ·
20 否则是没有输出的
21 sc.close();//关闭流并释放
22 }

```

14.1.2 优先队列

```

1 PriorityQueue queue = new PriorityQueue( 1,
2     new Comparator(){
3         public int compare( Point a, Point b ){
4             if( a.x < b.x || a.x == b.x && a.y < b.y )
5                 return -1;
6             else if( a.x == b.x && a.y == b.y )
7                 return 0;
8             else return 1;
9         }
10    });

```

14.1.3 Map

```

1 Map map = new HashMap();
2 map.put("sa","dd");
3 String str = map.get("sa").toString();
4
5 for(Object obj : map.keySet()){
6     Object value = map.get(obj );
7 }

```

14.1.4 sort

```

1 static class cmp implements Comparator{
2     public int compare(Object o1,Object o2){
3         BigInteger b1=(BigInteger)o1;
4         BigInteger b2=(BigInteger)o2;
5         return b1.compareTo(b2);
6     }
7 }
8 public static void main(String[] args)
9     throws IOException{
10     Scanner cin = new Scanner(System.in);
11     int n;
12     n=cin.nextInt();
13     BigInteger[] seg = new BigInteger[n];
14     for (int i=0;i<n;i++)
15         seg[i]=cin.nextBigInteger();
16     Arrays.sort(seg,new cmp());

```

14.2 python heap

```

1 import heapq
2
3 heap = [7,1,2,2]
4 heapq.heapify(heap)
5 print(heap) # [1, 2, 2, 7]
6 heapq.heappush(heap, 5)
7 print(heap) # [1, 2, 2, 7, 5]
8 print(heapq.heappop(heap)) # 1
9 print(heap) # [2, 2, 5, 7]

```

14.3 python input

```

1 ans = sum(map(float, input().split()))
2 # input: 1.1 2.2 3.3 4.4 5.5
3 print(ans) # 16.5
4
5 (n, m) = map(int, input().split()) # 300 200
6 print(n * m) # 60000
7
8 Arr = list(map(int, input().split()))
9 # input: 1 2 3 4 5
10 print(Arr) # [1, 2, 3, 4, 5]

```

14.4 python output

```

1 hello = 'Hello'
2 world = 7122
3 print(f'{hello} {world}') # Hello 7122
4
5 import math
6 print(f'PI is approximately {math.pi:.3f}.')
7 # PI is approximately 3.142.
8
9 print('AAA {} BBB "{}!"'.format('Jin', 'Kela'))
10 # AAA Jin BBB "Kela!"
11
12 hello = 'hello, world\n'
13 hellos = repr(hello)
14 print(hellos) # 'hello, world\n'
15
16 x = 32.5
17 y = 40000
18 print(repr((x, y, ('spam', 'eggs'))))
19 # "(32.5, 40000, ('spam', 'eggs'))"
20
21 x = 7
22 print(eval('3 * x')) # 21

```

15 zformula

15.1 formula

15.1.1 Pick 公式

給定頂點坐標均是整點的簡單多邊形，面積 = 內部格點數 + 邊上格點數/2-1

15.1.2 圖論

- 對於平面圖： $F = E - V + C + 1$ ， C 是連通分量數
- 對於平面圖： $E \leq 3V - 6$
- 對於連通圖 G ，最大獨立點集的大小設為 $I(G)$ ，最大匹配大小設為 $M(G)$ ，最小點覆蓋設為 $Cv(G)$ ，最小邊覆蓋設為 $Ce(G)$ 。對於任意連通圖：

$$(a) I(G) + Cv(G) = |V|$$

$$(b) M(G) + Ce(G) = |V|$$

- 對於連通二分圖：

$$(a) I(G) = Cv(G)$$

$$(b) M(G) = Ce(G)$$

- 最大權閉合圖：

$$(a) C(u, v) = \infty, (u, v) \in E$$

$$(b) C(S, v) = W_v, W_v > 0$$

$$(c) C(v, T) = -W_v, W_v < 0$$

$$(d) ans = \sum_{W_v > 0} W_v - flow(S, T)$$

- 最大密度子圖：

$$(a) \text{求 } \max \left(\frac{W_e + W_v}{|V|} \right), e \in E', v \in V'$$

$$(b) U = \sum_{v \in V} 2W_v + \sum_{e \in E} W_e$$

$$(c) C(u, v) = W_{(u, v)}, (u, v) \in E \cdot \text{雙向邊}$$

$$(d) C(S, v) = U, v \in V$$

$$(e) D_u = \sum_{(u, v) \in E} W_{(u, v)}$$

$$(f) C(v, T) = U + 2g - D_v - 2W_v, v \in V$$

$$(g) \text{二分搜 } g:$$

$$l = 0, r = U, \text{eps} = 1/n^2$$

$$\text{if}((U \times |V| - flow(S, T))/2 > 0) \text{ } l = mid$$

$$\text{else } r = mid$$

$$(h) ans = \min_cut(S, T)$$

$$(i) |E| = 0 \text{ 要特殊判斷}$$

- 弦圖：

$$(a) \text{點數大於 } 3 \text{ 的環都要有一條弦}$$

$$(b) \text{完美消除序列從後往前依次給每個點染色 · 給每個點染上可以染的最小顏色}$$

$$(c) \text{最大團大小 = 色數}$$

$$(d) \text{最大獨立集: 完美消除序列從前往後能選就選}$$

$$(e) \text{最小團覆蓋: 完美消除序列從後往前能選就選}$$

$$(f) \text{區間圖是弦圖}$$

$$(g) \text{區間圖的完美消除序列: 將區間按造又端點由小到大排序}$$

$$(h) \text{區間圖染色: 用線段樹做}$$

15.1.3 dinic 特殊圖複雜度

- 單位流： $O\left(\min\left(V^{3/2}, E^{1/2}\right)E\right)$
- 二分圖： $O\left(V^{1/2}E\right)$

15.1.4 0-1 分數規劃

$$x_i = \{0, 1\} \cdot x_i \text{ 可能會有其他限制} \cdot \text{求 } \max\left(\frac{\sum B_i x_i}{\sum C_i x_i}\right)$$

- $D(i, g) = B_i - g \times C_i$
- $f(g) = \sum D(i, g) x_i$
- $f(g) = 0$ 時 g 為最佳解 · $f(g) < 0$ 沒有意義
- 因為 $f(g)$ 單調可以二分搜 g
- 或用 Dinkelbach 通常比較快

```

1 binary_search(){
2     while(r-l>eps){
3         g=(l+r)/2;
4         for(i:所有元素)D[i]=B[i]-g*C[i]; //D(i,g)
5         找出一組合法x[i]使f(g)最大;
6         if(f(g)>0) l=g;
7         else r=g;
8     }
9     Ans = r;
10 }
11 Dinkelbach(){
12     g=任意狀態(通常設為0);
13     do{
14         Ans=g;
15         for(i:所有元素)D[i]=B[i]-g*C[i]; //D(i,g)
16         找出一組合法x[i]使f(g)最大;
17         p=0, q=0;
18         for(i:所有元素)
19             if(x[i])p+=B[i], q+=C[i];
20         g=p/q; //更新解 · 注意q=0的情況
21     }while(abs(Ans-g)>EPS);
22     return Ans;
23 }

```

15.1.5 學長公式

- $\sum_{d|n} \phi(n) = n$
- $g(n) = \sum_{d|n} f(d) \Rightarrow f(n) = \sum_{d|n} \mu(d) \times g(n/d)$
- Harmonic series $H_n = \ln(n) + \gamma + 1/(2n) - 1/(12n^2) + 1/(120n^4)$
- $\gamma = 0.57721566490153286060651209008240243104215$
- 格雷碼： $n \oplus (n >> 1)$
- $SG(A + B) = SG(A) \oplus SG(B)$
- 選轉矩陣 $M(\theta) = \begin{pmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{pmatrix}$

15.1.6 基本數論

- $\sum_{d|n} \mu(n) = [n == 1]$
- $g(m) = \sum_{d|m} f(d) \Leftrightarrow f(m) = \sum_{d|m} \mu(d) \times g(m/d)$
- $\sum_{i=1}^n \sum_{j=1}^m \text{互質數量} = \sum \mu(d) \lfloor \frac{n}{d} \rfloor \lfloor \frac{m}{d} \rfloor$
- $\sum_{i=1}^n \sum_{j=1}^m \text{lcm}(i, j) = n \sum_{d|n} d \times \phi(d)$

15.1.7 排組公式

1. k 卡特蘭 $\frac{C_n^{kn}}{n(k-1)+1} \cdot C_m^n = \frac{n!}{m!(n-m)!}$
2. $H(n, m) \cong x_1 + x_2 \dots + x_n = k, num = C_k^{n+k-1}$
3. Stirling number of 2^{nd} , n 人分 k 組方法數目

- (a) $S(0, 0) = S(n, n) = 1$
- (b) $S(n, 0) = 0$
- (c) $S(n, k) = kS(n-1, k) + S(n-1, k-1)$

4. Bell number, n 人分任意多組方法數目

- (a) $B_0 = 1$
- (b) $B_n = \sum_{i=0}^n S(n, i)$
- (c) $B_{n+1} = \sum_{k=0}^n C_k^n B_k$
- (d) $B_{p+n} \equiv B_n + B_{n+1} \pmod{p}$, p is prime
- (e) $B_{p^m+n} \equiv mB_n + B_{n+1} \pmod{p}$, p is prime
- (f) From $B_0 : 1, 1, 2, 5, 15, 52, 203, 877, 4140, 21147, 115975$

5. Derangement, 錯排, 沒有人在自己位置上

- (a) $D_n = n!(1 - \frac{1}{1!} + \frac{1}{2!} - \frac{1}{3!} \dots + (-1)^n \frac{1}{n!})$
- (b) $D_n = (n-1)(D_{n-1} + D_{n-2}), D_0 = 1, D_1 = 0$
- (c) From $D_0 : 1, 0, 1, 2, 9, 44, 265, 1854, 14833, 133496$

6. Binomial Equality

- (a) $\sum_k \binom{r}{m+k} \binom{s}{n-k} = \binom{r+s}{m+n}$
- (b) $\sum_k \binom{l}{m+k} \binom{s}{n+k} = \binom{l+s}{l-m+n}$
- (c) $\sum_k \binom{l}{m+k} \binom{s+k}{n} (-1)^k = (-1)^{l+m} \binom{s-m}{n-l}$
- (d) $\sum_{k \leq l} \binom{l-k}{m} \binom{s-n}{k-n} (-1)^k = (-1)^{l+m} \binom{s-m-1}{l-n-m}$
- (e) $\sum_{0 \leq k \leq l} \binom{l-k}{m} \binom{q+k}{n} = \binom{l+q+1}{m+n+1}$
- (f) $\binom{r}{k} = (-1)^k \binom{k-r-1}{k}$
- (g) $\binom{r}{m} \binom{m}{k} = \binom{r}{k} \binom{r-k}{m-k}$
- (h) $\sum_{k \leq n} \binom{r+k}{k} = \binom{r+n+1}{n}$
- (i) $\sum_{0 \leq k \leq n} \binom{k}{m} = \binom{n+1}{m+1}$
- (j) $\sum_{k \leq m} \binom{m+r}{k} x^k y^{m-k} = \sum_{k \leq m} \binom{-r}{k} (-x)^k (x+y)^{m-k}$

7. 模擬賽遇到的

- (a) 對於一個長度為 L 的環, 使用 k 種顏色進行相鄰不同色的著色, 其著色方法數的公式為:

$$P(L, k) = (k-1)^L + (-1)^L (k-1)$$

15.1.8 冪次, 冪次和

1. $a^{b \% a} P = a^{b \% \varphi(p) + \varphi(p)}, b \geq \varphi(p)$
2. $1^3 + 2^3 + 3^3 + \dots + n^3 = \frac{n^4}{4} + \frac{n^2}{2} + \frac{n^2}{4}$
3. $1^4 + 2^4 + 3^4 + \dots + n^4 = \frac{n^5}{5} + \frac{n^4}{2} + \frac{n^3}{3} - \frac{n}{30}$
4. $1^5 + 2^5 + 3^5 + \dots + n^5 = \frac{n^6}{6} + \frac{n^5}{2} + \frac{5n^4}{12} - \frac{n^2}{12}$

5. $0^k + 1^k + 2^k + \dots + n^k = P(k), P(k) = \frac{(n+1)^{k+1} - \sum_{i=0}^{k-1} C_i^{k+1} P(i)}{k+1}, P(0) = n+1$
6. $\sum_{k=0}^{m-1} k^n = \frac{1}{n+1} \sum_{k=0}^n C_k^{n+1} B_k m^{n+1-k}$
7. $\sum_{j=0}^m C_j^{m+1} B_j = 0, B_0 = 1$
8. 除了 $B_1 = -1/2$, 剩下的奇數項都是 0
9. $B_2 = 1/6, B_4 = -1/30, B_6 = 1/42, B_8 = -1/30, B_{10} = 5/66, B_{12} = -691/2730, B_{14} = 7/6, B_{16} = -3617/510, B_{18} = 43867/798, B_{20} = -174611/330,$

15.1.9 Burnside's lemma

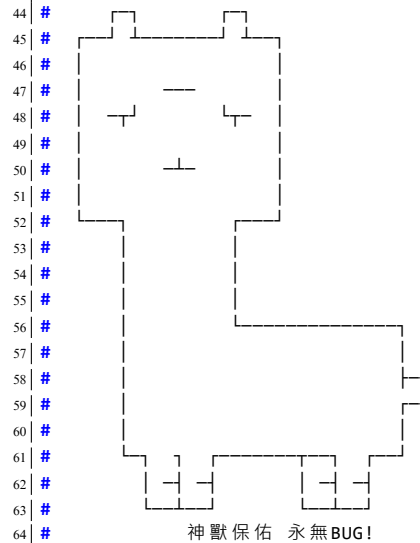
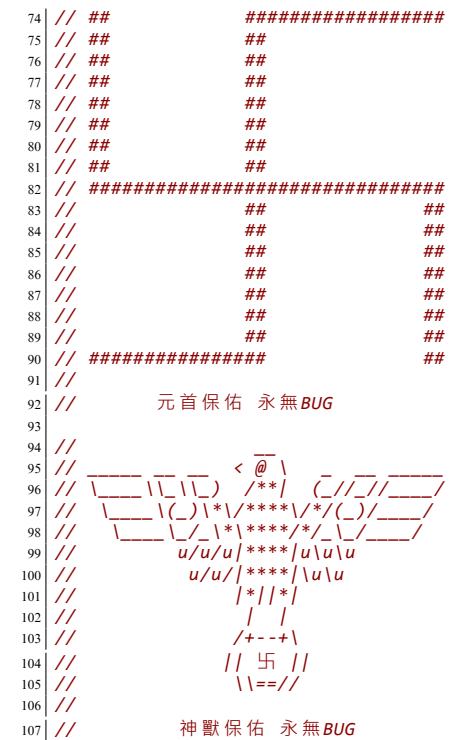
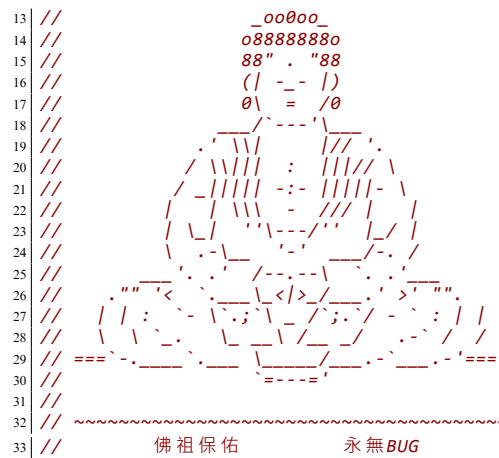
1. $|X/G| = \frac{1}{|G|} \sum_{g \in G} |X^g|$
2. $X^g = t^{c(g)}$
3. G 表示有幾種轉法, X^g 表示在那種轉法下, 有幾種是會保持對稱的, t 是顏色數, $c(g)$ 是循環節不動的面數。
4. 正立方體塗三顏色, 轉 0 有 3^6 個元素不變, 轉 90 有 6 種, 每種有 3^3 不變, 180 有 $3 \times 3^4 \cdot 120(\text{角})$ 有 $8 \times 3^2 \cdot 180(\text{邊})$ 有 $6 \times 3^3 \cdot \text{全部}$ $\frac{1}{24} (3^6 + 6 \times 3^3 + 3 \times 3^4 + 8 \times 3^2 + 6 \times 3^3) = 57$

15.1.10 Count on a tree

1. Rooted tree: $s_{n+1} = \frac{1}{n} \sum_{i=1}^n (i \times a_i \times \sum_{j=1}^{\lfloor n/i \rfloor} a_{n+1-i \times j})$
2. Unrooted tree:
 - (a) Odd: $a_n - \sum_{i=1}^{n/2} a_i a_{n-i}$
 - (b) Even: $Odd + \frac{1}{2} a_{n/2} (a_{n/2} + 1)$
3. Spanning Tree
 - (a) 完全圖 $n^n - 2$
 - (b) 一般圖 (Kirchhoff's theorem) $M[i][i] = \text{degree}(V_i), M[i][j] = -1, \text{if have } E(i, j), 0 \text{ if no edge. delete any one row and col in } A, \text{ans} = \det(A)$

16 Интернационал

16.1 保佑



ACM ICPC Team Reference - CCU_BABABOY

Contents

1 Computational Geometry	1	3 Flow	4	6.16 質因數分解	10	12 dp 優化	15
1.1 basic	1	3.1 dinic	4	7 String	10	12.1 2D1D 優化	15
1.2 convex-hull	1	3.2 dinic flow decomposition	4	7.1 ac 自動機	10	12.2 CHT	16
1.3 Line Segment Intersection	1	3.3 Edmonds-Karp	5	7.2 hash	11	12.3 SOS	16
1.4 Point in Polygon	1	3.4 Min Cost Max-Flow	5	7.3 KMP	11	12.4 分治優化	16
1.5 動態開點李超樹	1	3.5 MinCut	5	7.4 manacher	11	13 other	16
1.6 半平面交集	1	4 Graph	5	7.5 suffix-array	11	13.1 WhatDay	16
1.7 最近點對	2	4.1 bellman-ford	5	7.6 Zfunction	11	13.2 上下最大正方形	16
1.8 李超線段樹	2	4.2 dijk	5	8 Tarjan	11	13.3 最大矩形	16
1.9 矩形覆蓋面積	2	4.3 Euler-Path	5	8.1 dominator tree	11	14 other language	16
2 Data Structure	2	4.4 FunctionalGraph	5	8.2 EBCC	12	14.1 java	16
2.1 DSU-rollback	2	4.5 SCC	6	8.3 SCC	12	14.1.1 文件操作	16
2.2 FHQ-treap	2	4.6 tarjan scc	6	8.4 tnfsbh017 2 sat	12	14.1.2 優先隊列	17
2.3 persistent-segment-tree	3	4.7 線段樹優化建圖	6	8.5 vbcc	12	14.1.3 Map	17
2.4 segment tree lazytag	3	5 Linear Programming	7	8.6 橋連通分量	13	14.1.4 sort	17
2.5 trie	3	5.1 simplex	7	8.7 雙連通分量 & 割點	13	14.2 python heap	17
2.6 Weighted DSU	4	6 Number Theory	7	9 Tree Problem	13	14.3 python input	17
2.7 動態開點線段樹	4	6.1 basic	7	9.1 centroid decomposition	13	14.4 python output	17
		6.2 bit set	8	9.2 find-centroid	14	15 zformula	17
		6.3 cantor expansion	8	9.3 hld	14	15.1 formula	17
		6.4 FFT	8	9.4 lca	14	15.1.1 Pick 公式	17
		6.5 FFT2	8	9.5 tree-hash	14	15.1.2 圖論	17
		6.6 find real root	8	10 compare	15	15.1.3 dinic 特殊圖複雜度	17
		6.7 FWT	8	10.1 check	15	15.1.4 0-1 分數規劃	17
		6.8 LinearCongruence	9	10.2 gen	15	15.1.5 學長公式	17
		6.9 Lucas	9	11 default	15	15.1.6 基本數論	17
		6.10 Matrix	9	11.1 debug	15	15.1.7 排組公式	18
		6.11 MillerRobin	9	11.2 ext	15	15.1.8 冪次, 冪次和	18
		6.12 NTT	9	11.3 IncStack	15	15.1.9 Burnside's lemma	18
		6.13 Simpson	10	11.4 input	15	15.1.10 Count on a tree	18
		6.14 外星模運算	10			16 Интернационал	18
		6.15 數位統計	10			16.1 保佑	18

ACM

ICPC Judge Test - CCU_BABABOY

C++ Resource Test

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 namespace system_test {
5
6 const size_t KB = 1024;
7 const size_t MB = KB * 1024;
8 const size_t GB = MB * 1024;

```

```

9 size_t block_size, bound;
10 void stack_size_dfs(size_t depth = 1) {
11     if (depth >= bound)
12         return;
13     int8_t ptr[block_size]; // 若無法編譯將
14                             // block_size 改成常數
15     memset(ptr, 'a', block_size);
16     cout << depth << endl;
17     stack_size_dfs(depth + 1);
18 }
19
20 void stack_size_and_runtime_error(size_t
21     block_size, size_t bound = 1024) {
22     system_test::block_size = block_size;
23     system_test::bound = bound;
24     stack_size_dfs();
25 }
26
27 double speed(int iter_num) {
28     const int block_size = 1024;
29     volatile int A[block_size];
30     auto begin = chrono::high_resolution_clock
31         ::now();
32     while (iter_num--)
33         for (int j = 0; j < block_size; ++j)
34             A[j] += j;
35     auto end = chrono::high_resolution_clock::
36         now();

```

```

34     chrono::duration<double> diff = end -
35         begin;
36     return diff.count();
37 }
38
39 void runtime_error_1() {
40     // Segmentation fault
41     int *ptr = nullptr;
42     *(ptr + 7122) = 7122;
43 }
44
45 void runtime_error_2() {
46     // Segmentation fault
47     int *ptr = (int *)memset;
48     *ptr = 7122;
49 }
50
51 void runtime_error_3() {
52     // munmap_chunk(): invalid pointer
53     int *ptr = (int *)memset;
54     delete ptr;
55 }
56
57 void runtime_error_4() {
58     // free(): invalid pointer
59     int *ptr = new int[7122];
60     ptr += 1;
61     delete[] ptr;

```

```

62
63 void runtime_error_5() {
64     // maybe illegal instruction
65     int a = 7122, b = 0;
66     cout << (a / b) << endl;
67 }
68
69 void runtime_error_6() {
70     // floating point exception
71     volatile int a = 7122, b = 0;
72     cout << (a / b) << endl;
73 }
74
75 void runtime_error_7() {
76     // call to abort.
77     assert(false);
78 }
79
80 } // namespace system_test
81
82 #include <sys/resource.h>
83 void print_stack_limit() { // only work in
84     Linux
85     struct rlimit l;
86     getrlimit(RLIMIT_STACK, &l);
87     cout << "stack_size = " << l.rlim_cur << "
88         byte" << endl;

```